République du Bénin

Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

Université Nationale des Sciences, Technologies,
Ingénierie et Mathématiques d'Abomey (UNSTIM)

École Nationale des Génies Mathématiques et Modélisation (ENSGMM)

Modélisation Haute Performance de l'Infection par le VIH-SIDA

Dynamique de Cohorte Clinique, Émergence de Résistances aux Antirétroviraux et Analyse Comparative des Bibliothèques BLAS, LAPACK et Intel MKL en Python

Équipe de projet – Groupe 5

ADOGOUN Déo Gracias Mahugnon

BIAHOU Ahossou J. Marie-Clémenceau

SYLLA IMOROU Izou Dine

Dépôt GitHub officiel (Code Source, Benchmarks & Reproductibilité) :
`https://github.com/JuniorB573/OCS-Groupe5`

Promotion : ENSGMM1 • **UE :** Outils de calcul scientifique

Année académique : 2026–2027

Sous la direction de : Dr. Carlos AGOSSOU

Table des matières

Introduction Générale **3**

1 Fondements et Architecture de BLAS **5**
1.1 Origine, Philosophie et Taxonomie 5
1.2 Hiérarchie des Trois Niveaux et Intensité Arithmétique 5
1.3 Le Rôle Crucial de BLAS 3 et l'Algorithme par Blocs (`DGEMM`) 5

2 LAPACK : Algèbre Linéaire Dense Avancée **7**
2.1 Motivation et Rupture avec LINPACK/EISPACK 7
2.2 La Factorisation de Cholesky (`DPOTRF` / `DPOTRS`) 7
2.3 Le Solveur Spectral Divide & Conquer (`DSYEVD`) 7

3 Intel oneAPI MKL et Optimisations Matérielles **8**
3.1 Architecture Interne d'Intel MKL 8
3.2 Vectorisation SIMD (AVX2 et AVX-512) 8
3.3 Intel MKL Vector Math Library (VML) 8

4 Installation, Configuration et Utilisation des Bibliothèques **9**
4.1 Installation de la Pile Scientifique avec Intel MKL 9
4.2 Inspection et Détection Dynamique du Backend Actif 9
4.3 Gestion du Multithreading et Variables d'Environnement 10
4.4 Contraintes Mémoire : Ordre C vs Ordre Fortran (Zéro-Copie) 10
4.5 Appel Direct des Micro-Noyaux BLAS et LAPACK depuis SciPy 10

5 Le Modèle Biomédical de Cohorte VIH **12**
5.1 Biopathologie du VIH et Émergence de Résistance 12
5.2 Variables d'État et Compartiments Physiologiques 12
5.3 Le Système d'Équations Différentielles Couplées 13

6 Formulation Matricielle Globale et Intégration RK4 **14**
6.1 Représentation Vectorisée de la Cohorte en Mémoire Continue 14
6.2 Scission Opératorielle et Formulation Matricielle Pure 14
6.3 Algorithme d'Intégration RK4 Matriciel 15

7 Analyse Pronostique et Résolution LAPACK **16**
7.1 Régression Clinique Multivariée Régularisée (Ridge) 16
7.2 Chaîne de Traitement Haute Performance LAPACK 16
7.3 Analyse en Composantes Principales (ACP) par `DSYEVD` 16

8 Résultats Médicaux et Analyse de la Dynamique Virale **18**
8.1 Trajectoires Cliniques de la Cohorte sous Trithérapie 18
8.2 Résultats de la Régression Pronostique Ridge (LAPACK) 18
8.3 Cartographie Spectrale par ACP LAPACK (`DSYEVD`) 19

9 Campagne de Benchmarks et Évaluation des Performances **20**
9.1 Protocole Expérimental et Environnement Matériel 20
9.2 Benchmark 1 : Simulation de Cohorte (Naïf vs BLAS 3 / MKL) 20
9.3 Benchmark 2 : Montée en Charge et Débit en GFLOPS 20
9.4 Benchmark 3 : Scalabilité Parallèle Multi-Cœurs OpenMP 22

10 Guide d'Exécution et Reproductibilité **23**
10.1 Philosophie de la Reproductibilité Computationnelle 23
10.2 Arborescence Normative du Projet 23
10.3 Protocole de Déploiement et d'Installation 24
10.3.1 1. Clonage du Répertoire Git 24
10.3.2 2. Instanciation de l'Environnement Virtuel 24
10.3.3 3. Installation des Dépendances Numériques 24
10.4 Protocole de Reproduction des Simulations et Expériences 24
10.4.1 1. Simulation Clinique de Cohorte (`scripts/run_simulation.py`) 24
10.4.2 2. Campagne de Benchmarks et Profilage (`scripts/run_benchmarks.py`) . 25
10.4.3 3. Validation Formelle par la Suite de Tests (`pytest`) 25
10.5 Contrôle du Déterminisme et Reproductibilité Matérielle 25
10.6 Compilation des Livrables LaTeX 26

Conclusion Générale et Perspectives **27**

A Noyaux Numériques de Modélisation et Résolution **28**
A.1 Modèle de Cohorte Vectorisé (cohort_model.py) 28
A.2 Modèle Scalaire Naïf (naive_model.py) 33
A.3 Analyse Pronostique LAPACK (lapack_analysis.py) 35

B Bancs d'Essai, Benchmarks et Tests de Validation **38**
B.1 Moteur de Benchmarks (benchmark.py) 38
B.2 Script de Lancement des Benchmarks (run_benchmarks.py) 44
B.3 Suite de Tests d'Équivalence (test_equivalence.py) 48

Introduction Générale

Résumé Exécutif & Enjeux Scientifiques

Ce rapport présente la conception, l'implémentation et l'analyse de performance d'un modèle biomédical de haute fidélité simulant l'infection par le Virus de l'Immunodéficience Humaine (VIH) à l'échelle d'une **cohorte clinique de patients sous trithérapie antirétrovirale (ART)**.
Le Défi du Calcul Haute Performance (HPC) : Au lieu de restreindre la modélisation à un cas scalaire artificiel à trois équations, nous formulons la dynamique globale de milliers de patients sous forme d'un opérateur différentiel matriciel continu en mémoire. Nous démontrons de manière expérimentale et théorique la supériorité absolue des micro-noyaux vectorisés **BLAS 3** (`DGEMM`, `DSYRK`), des factorisations directes **LAPACK** (`DPOTRF`, `DPOTRS`, `DSYEVD`) et des optimisations de la bibliothèque **Intel Math Kernel Library (MKL)**, offrant des accélérations d'un ordre de grandeur face aux méthodes classiques séquentielles et non vectorisées.

Contexte et Motivation. Le calcul scientifique moderne a cessé d'être limité par la seule fréquence brute d'horloge des processeurs. Depuis l'avènement des architectures superscalaires et multi-cœurs dotées d'unités de calcul vectoriel SIMD (AVX2, AVX-512), l'efficacité algorithmique dépend de la localité spatiale et temporelle des données en mémoire cache et de l'intensité arithmétique des opérations.

Dans le domaine biomédical, le suivi clinique des personnes vivant avec le VIH requiert la simulation simultanée de cohortes de plusieurs milliers de patients pour évaluer l'impact de l'observance thérapeutique et prédire l'émergence de souches virales mutantes résistantes. Résoudre ce problème par des boucles séquentielles ordinaires en Python pur constitue un goulot d'étranglement majeur. À l'inverse, projeter l'ensemble de la cohorte dans un espace tensoriel où chaque pas de temps se réduit à des multiplications matricielles `DGEMM` permet de saturer les capacités crêtes du processeur.

Objectifs Spécifiques du Projet :

1. **Étude Théorique des Bibliothèques de Base :** Exposer les fondements mathématiques et informatiques de BLAS (Niveaux 1, 2, 3), LAPACK (factorisations LU, Cholesky, décompositions spectrales) et Intel MKL.
2. **Formulation d'un Nouveau Modèle VIH Réaliste :** Modéliser rigoureusement les interactions entre lymphocytes T CD4+, cellules infectées, souche sauvage sensible et souche mutante résistante au sein d'une cohorte clinique hétérogène.
3. **Intégration Temporelle Matricielle :** Établir le schéma matriciel scindé $\dot{\mathbf{Y}} = \mathbf{S} + \mathbf{Y}\mathbf{M}^T + \mathbf{\Phi}\mathbf{A}^T$ résolu par un algorithme de Runge-Kutta d'ordre 4 (RK4) vectorisé.
4. **Analyse Pronostique Haute Performance (LAPACK) :** Identifier les biomarqueurs d'échec thérapeutique par régression régularisée de Ridge résolue via Cholesky (`DPOTRF` / `DPOTRS`) et cartographier les profils patients par ACP spectrale (`DSYEVD`).

5. **Campagne de Benchmarks et Validation Rigoureuse :** Mesurer les temps d'exécution, facteurs d'accélération (Speedup), débits en GFLOPS et scalabilité multithread OpenMP en confrontant la bibliothèque Intel aux approches naïves.

Organisation du Rapport :

— **Chapitres 1 à 3 :** Fondements théoriques de BLAS (Niveaux 1, 2, 3 et intensité arithmétique), LAPACK (factorisations LU, Cholesky, décompositions spectrales) et Intel oneAPI MKL (SIMD AVX2/AVX-512, bibliothèque vectorielle VML).

— **Chapitre 4 : Installation, Configuration et Utilisation des Bibliothèques** (déploiement Conda/pip, détection dynamique des micro-noyaux avec `threadpoolctl`, contrôle des threads OpenMP et stockage mémoire Fortran zéro-copie).

— **Chapitres 5 à 7 :** Modélisation biomédicale de cohorte VIH, formulation tensorielle matricielle scindée $\dot{\mathbf{Y}} = \mathbf{S} + \mathbf{Y}\mathbf{M}^T + \mathbf{\Phi}\mathbf{A}^T$, intégration Runge-Kutta 4 (BLAS 3) et analyse pronostique LAPACK (Cholesky et ACP).

— **Chapitres 8 et 9 :** Résultats médicaux cliniques et campagne de benchmarks comparatifs (confrontation méthodique avec les méthodes lentes naïves).

— **Chapitre 10 : Guide d'Exécution et Reproductibilité** (protocoles d'exécution pas à pas, reproduction de la simulation, des benchmarks et de la validation formelle pytest).

— **Annexes A et B :** Codes sources exhaustifs documentés du simulateur, des modèles naïfs, des solveurs LAPACK, du banc d'essai et des tests unitaires.

Chapitre 1

Fondements et Architecture de BLAS

1.1 Origine, Philosophie et Taxonomie

La bibliothèque **BLAS** (*Basic Linear Algebra Subprograms*), initiée en 1979 par Lawson, Hanson, Kincaid et Krogh, constitue le socle universel du calcul matriciel dense. Sa standardisation a permis de dissocier formellement la spécification mathématique de l'implémentation matérielle : les développeurs écrivent leurs algorithmes en invoquant des primitives invariantes, tandis que les constructeurs de puces électroniques (Intel, AMD, ARM) fournissent des binaires hautement spécialisés exploitant les registres et micro-architectures internes.

1.2 Hiérarchie des Trois Niveaux et Intensité Arithmétique

L'efficacité d'un algorithme numérique sur une architecture moderne est gouvernée par son **intensité arithmétique** I, définie comme le ratio entre le nombre d'opérations en virgule flottante (FLOPs) et le volume de données transféré depuis la mémoire vive (en octets) :

$$I = \frac{\text{Opérations (FLOPs)}}{\text{Transferts Mémoire (Octets)}} \tag{1.1}$$

Table 1.1 – Classification hiérarchique des trois niveaux de BLAS et intensité arithmétique.

Niveau	Opération	Données	FLOPs	Intensité I	Routines Phares
1	Vecteur-Vecteur	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(1)$ (Mémoire)	`DAXPY` ($y \leftarrow \alpha x + y$), `DDOT`, `DNRM2`
2	Matrice-Vecteur	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$ (Mémoire)	`DGEMV` ($y \leftarrow \alpha Ax + \beta y$), `DGER`, `DTRSV`
3	Matrice-Matrice	$\mathcal{O}(n^2)$	$\mathcal{O}(n^3)$	$\mathcal{O}(n)$ (Calcul)	`DGEMM` ($C \leftarrow \alpha AB + \beta C$), `DSYRK`, `DTRMM`

1.3 Le Rôle Crucial de BLAS 3 et l'Algorithme par Blocs (`DGEMM`)

Tandis que les Niveaux 1 et 2 sont bridés par la bande passante du bus mémoire (*Memory-Bound*), le Niveau 3 présente une complexité volumique en $\mathcal{O}(n^3)$ pour seulement $\mathcal{O}(n^2)$ données lues. Il devient ainsi possible de réutiliser les données stockées dans les caches ultra-rapides L1, L2 et L3 du processeur grâce à des techniques de partitionnement par blocs (*Cache Blocking* et *Loop Tiling*).

La routine centrale `DGEMM` réalise la mise à jour générale :

$$\mathbf{C} \leftarrow \alpha\, \mathrm{op}(\mathbf{A})\, \mathrm{op}(\mathbf{B}) + \beta \mathbf{C} \tag{1.2}$$

où $\mathrm{op}(\mathbf{X}) \in \{\mathbf{X}, \mathbf{X}^T, \mathbf{X}^H\}$. Dans notre modèle de cohorte VIH, chaque pas d'intégration temporelle projette l'ensemble des patients via deux appels successifs à `DGEMM`, garantissant une saturation quasi-totale des unités de calcul du microprocesseur.

Chapitre 2

LAPACK : Algèbre Linéaire Dense Avancée

2.1 Motivation et Rupture avec LINPACK/EISPACK

Développée au début des années 1990 sous la direction de Jack Dongarra et Jim Demmel, **LAPACK** (*Linear Algebra PACKage*) a été conçue pour surmonter les limitations de LINPACK sur les architectures dotées de hiérarchies de caches complexes. LAPACK restructure systématiquement les algorithmes de factorisation (LU, QR, Cholesky, SVD) afin d'exprimer plus de 85% du travail numérique sous forme d'appels à des blocs de Niveau 3 de BLAS (`DGEMM`, `DTRSM`).

2.2 La Factorisation de Cholesky (`DPOTRF` / `DPOTRS`)

Pour résoudre un système linéaire symétrique défini positif issu des équations normales de régression clinique :

$$\mathbf{G}\mathbf{w} = \mathbf{b}, \quad \text{avec } \mathbf{G} = \mathbf{X}^T\mathbf{X} + \lambda\mathbf{I} \tag{2.1}$$

la factorisation de Cholesky décompose la matrice de Gram sous forme triangulaire :

$$\mathbf{G} = \mathbf{L}\mathbf{L}^T \tag{2.2}$$

où $\mathbf{L}$ est une matrice triangulaire inférieure à coefficients diagonaux strictement positifs.

Proposition 2.1 (Coût Numérique et Stabilité de Cholesky)**.** *La factorisation de Cholesky (`DPOTRF`) requiert exactement $\frac{1}{3}p^3 + \mathcal{O}(p^2)$ opérations en virgule flottante, soit deux fois moins de calculs qu'une factorisation LU classique ($\frac{2}{3}p^3$) et six fois moins qu'une inversion matricielle explicite par élimination de Gauss-Jordan ($2p^3$). De surcroît, elle est inconditionnellement stable sans pivotage.*

La résolution subséquente (`DPOTRS`) s'effectue en deux étapes triangulaires rapides $\mathcal{O}(p^2)$ par descente puis remontée :

$$\mathbf{L}\mathbf{z} = \mathbf{b} \quad \text{(Descente)}, \qquad \mathbf{L}^T\mathbf{w} = \mathbf{z} \quad \text{(Remontée)} \tag{2.3}$$

2.3 Le Solveur Spectral Divide & Conquer (`DSYEVD`)

Pour l'analyse en composantes principales des biomarqueurs de patients, LAPACK dispose de la routine `DSYEVD`. Contrairement à l'algorithme QR séquentiel classique (`DSYEV`), `DSYEVD` implémente le paradigme de division et conquête (*Divide and Conquer*) de Cuppen. Il découpe récursivement la matrice tridiagonale intermédiaire en sous-problèmes indépendants résolus en parallèle, procurant un gain d'accélération allant d'un facteur 3 à 8 sur les matrices denses.

Chapitre 3

Intel oneAPI MKL et Optimisations Matérielles

3.1 Architecture Interne d'Intel MKL

La bibliothèque **Intel Math Kernel Library (MKL)**, intégrée à l'écosystème Intel oneAPI, représente l'implémentation industrielle de référence pour l'architecture x86-64. Elle contient des versions ultra-optimisées en langage assembleur des spécifications BLAS et LAPACK, adaptées dynamiquement aux jeux d'instructions supportés par le processeur hôte (*CPU Dispatching*).

3.2 Vectorisation SIMD (AVX2 et AVX-512)

Les extensions vectorielles SIMD (*Single Instruction, Multiple Data*) permettent d'exécuter simultanément la même opération arithmétique sur un registre large contenant plusieurs scalaires double précision (64 bits) :

- **AVX2 (Registres 256 bits `ymm`) :** Traite 4 nombres en double précision par instruction.
- **AVX-512 (Registres 512 bits `zmm`) :** Traite 8 nombres en double précision par instruction.

Grâce à l'instruction FMA (*Fused Multiply-Add*) qui calcule $a \times b + c$ en un cycle d'horloge sans perte de précision intermédiaire, un processeur moderne exécutant deux unités FMA 512 bits délivre jusqu'à 16 FLOPs/cycle par cœur physique.

3.3 Intel MKL Vector Math Library (VML)

En complément de l'algèbre linéaire, Intel MKL intègre la bibliothèque vectorielle **VML**. Les fonctions transcendantes scalaires de base (`exp`, `log`, `pow`) sont notoirement coûteuses (souvent plus de 40 cycles CPU par évaluation). Les routines VML (`vdExp`, `vdLn`, `vdDiv`) calculent ces fonctions par blocs vectoriels de 1024 éléments avec une précision conforme aux normes IEEE-754, décuplant les débits de calcul des cinétiques non-linéaires d'infection.

Chapitre 4

Installation, Configuration et Utilisation des Bibliothèques

4.1 Installation de la Pile Scientifique avec Intel MKL

L'intégration des micro-noyaux Intel MKL au sein de l'environnement Python s'effectue via deux approches principales :

1. Via l'Écosystème Conda (Recommandé pour Intel MKL natif)

La distribution Anaconda/Miniconda fournit des binaires précompilés liant nativement NumPy et SciPy à Intel MKL :

```
# Création d'un environnement optimisé Intel MKL
conda create -n vih_hpc python=3.11 mkl mkl-service mkl_fft mkl_random
conda activate vih_hpc
conda install numpy scipy matplotlib threadpoolctl pytest
```

2. Via Pip et les Roues PyPI

Sous un environnement Python standard (CPython), l'installation s'effectue simplement via :

```
pip install numpy scipy matplotlib threadpoolctl pytest
```

Pour forcer l'usage spécifique des roues Intel MKL sous pip :

```
pip install mkl mkl-include mkl-service
pip install intel-numpy intel-scipy
```

4.2 Inspection et Détection Dynamique du Backend Actif

En calcul scientifique haute performance, il est impératif de vérifier dynamiquement quelle bibliothèque bas niveau est liée en mémoire vive ainsi que le nombre de threads physiques alloués :

```
import numpy as np
from threadpoolctl import threadpool_info, threadpool_limits

# 1. Vérification de la configuration d'accélération NumPy
print(np.show_config())

# 2. Inspection dynamique des bibliothèques actives en mémoire vive
info = threadpool_info()
for entry in info:
```

```
    print(f"Backend : {entry['user_api']} -> {entry['internal_api']} (Version {
        entry.get('version', 'N/A')})")
    print(f"Threads alloués : {entry['num_threads']}")
```

4.3 Gestion du Multithreading et Variables d'Environnement

Par défaut, les bibliothèques multithreadées détectent le nombre de cœurs logiques (Hyper-Threading). Afin d'éviter les contentions de cache L1/L2, il est recommandé de fixer le nombre de threads au nombre de cœurs physiques via les variables d'environnement système :

```
# Sous Linux / macOS (Bash)
export MKL_NUM_THREADS=4
export OMP_NUM_THREADS=4

# Sous Windows (PowerShell)
$env:MKL_NUM_THREADS = "4"
$env:OMP_NUM_THREADS = "4"
```

ou de façon dynamique et scopée en Python grâce à `threadpoolctl` :

```
with threadpool_limits(limits=4, user_api="blas"):
    # Calcul matriciel multithreadé optimisé
    times, trajectory = simulate_cohort_rk4(...)
```

4.4 Contraintes Mémoire : Ordre C vs Ordre Fortran (Zéro-Copie)

Les bibliothèques BLAS et LAPACK historiques sont conçues en Fortran et exigent des matrices organisées par colonnes (*Column-Major* ou `order="F"`), alors que le langage C et NumPy allouent par défaut en mémoire par lignes (*Row-Major* ou `order="C"`).

> **Règle d'or de performance :** Passer une matrice en ordre C à une routine LAPACK oblige SciPy à allouer une copie temporaire en mémoire pour transposer les données avant le calcul, puis à recopier le résultat, anéantissant l'efficacité temporelle. Il est capital d'instancier les matrices d'état avec `order="F"` ou via `np.asfortranarray` :

```
# Matrice garantie en mémoire continue Fortran (Zero-Copy)
A_fortran = np.asfortranarray(A, dtype=np.float64)
b_fortran = np.asfortranarray(b, dtype=np.float64)

# Appel direct du micro-noyau DGESV sans allocation intermédiaire
lu, piv, x, info = lapack.dgesv(A_fortran, b_fortran)
```

4.5 Appel Direct des Micro-Noyaux BLAS et LAPACK depuis SciPy

Les sous-modules `scipy.linalg.blas` et `scipy.linalg.lapack` fournissent un pont direct vers les fonctions compilées en C/Fortran sans aucun surcoût d'encapsulation :

```
from scipy.linalg import blas, lapack

# 1. Produit matriciel BLAS 3 : C = alpha * A @ B + beta * C
C = blas.dgemm(alpha=1.0, a=A_fortran, b=B_fortran)

# 2. Produit symétrique BLAS 3 : G = X.T @ X
```

```
G = blas.dsyrk(alpha=1.0, a=X_fortran, trans=1, lower=1)

# 3. Factorisation de Cholesky LAPACK : G = L @ L.T
L, info_potrf = lapack.dpotrf(G, lower=1)

# 4. Résolution triangulaire LAPACK : L @ L.T @ w = b
w, info_potrs = lapack.dpotrs(L, b_fortran, lower=1)

# 5. Décomposition spectrale Divide & Conquer : C @ v = lambda * v
w_eig, v_eig, info = lapack.dsyevd(Cov_matrix, compute_v=1, lower=1)
```

Synthèse du Chapitre 4 : Pratiques Clés en Python HPC

1. **Liaison Bas Niveau :** Vérifier systématiquement avec `threadpool_info()` et `np.show_config()` que NumPy/SciPy exploitent bien les bibliothèques multithreadées optimisées (Intel MKL ou OpenBLAS).
2. **Agencement Fortran Zéro-Copie :** Utiliser rigoureusement `order="F"` ou `np.asfortranarray` pour transmettre des blocs mémoires continus aux micro-noyaux BLAS/LAPACK sans recopie intermédiaire.
3. **Appel Direct des Micro-Noyaux :** Privilégier `scipy.linalg.blas` (`dgemm`, `dsyrk`) et `scipy.linalg.lapack` (`dpotrf`, `dsyevd`, `dgesv`) face aux fonctions génériques pour un contrôle total des allocations et des performances.
4. **Contrôle des Threads :** Calibrer `MKL_NUM_THREADS` ou `threadpool_limits` sur les cœurs physiques réels pour éviter les contentions de cache L1/L2.

Chapitre 5

Le Modèle Biomédical de Cohorte VIH

5.1 Biopathologie du VIH et Émergence de Résistance

Le Virus de l'Immunodéficience Humaine cible spécifiquement les lymphocytes T auxiliaires porteurs du récepteur CD4, cellules maîtresses de la coordination immunitaire adaptative. En l'absence de traitement, la destruction continue des CD4+ entraîne un effondrement immunitaire conduisant au stade SIDA.

La prise en charge thérapeutique repose sur la **trithérapie antirétrovirale combinée (ART)**, associant plusieurs classes médicamenteuses (inhibiteurs de rétro-transcriptase, d'intégrase et de protéase). Toutefois, la transcriptase inverse virale commet des erreurs fréquentes de réplication (taux de mutation $\mu \approx 10^{-4}$ par paire de bases). Sous traitement sous-optimal ou en cas d'inobservance thérapeutique, des variants porteurs de mutations de résistance émergent, échappent à l'inhibition chimique et provoquent un **rebond virologique**.

5.2 Variables d'État et Compartiments Physiologiques

Pour chaque patient $k \in \{1, \ldots, N\}$ de la cohorte, l'état clinique est modélisé par 5 variables d'état :

— $T_k(t)$: Concentration de lymphocytes T CD4+ non infectés (cellules/μL) ;
— $I_{s,k}(t)$: Cellules infectées par la souche sauvage sensible (cellules/μL) ;
— $I_{r,k}(t)$: Cellules infectées par la souche mutante résistante (cellules/μL) ;
— $V_{s,k}(t)$: Charge virale plasmatique de la souche sauvage (virions/mL) ;
— $V_{r,k}(t)$: Charge virale plasmatique de la souche mutante résistante (virions/mL).

5.3 Le Système d'Équations Différentielles Couplées

Le système non-linéaire régissant le patient k s'écrit :

$$\begin{cases} \dfrac{dT_k}{dt} = s_k - d\,T_k - (1-\eta_k)\beta_s T_k V_{s,k} - (1-\eta_{r,k})\beta_r T_k V_{r,k} \\ \dfrac{dI_{s,k}}{dt} = (1-\mu)(1-\eta_k)\beta_s T_k V_{s,k} - \delta\, I_{s,k} \\ \dfrac{dI_{r,k}}{dt} = \mu(1-\eta_k)\beta_s T_k V_{s,k} + (1-\eta_{r,k})\beta_r T_k V_{r,k} - \delta\, I_{r,k} \\ \dfrac{dV_{s,k}}{dt} = p\, I_{s,k} - c\, V_{s,k} \\ \dfrac{dV_{r,k}}{dt} = p\, I_{r,k} - c\, V_{r,k} \end{cases} \tag{5.1}$$

où l'efficacité résiduelle de la multithérapie sur le variant résistant s'exprime par :

$$\eta_{r,k} = \max\left(0,\, \eta_k - \Delta\eta\right), \quad \Delta\eta \approx 0.35 \tag{5.2}$$

illustrant le fait qu'une mutation confère un échappement partiel mais subit encore la pression des autres molécules de la trithérapie.

TABLE 5.1 – Paramètres biomédicaux du modèle intra-hôte de cohorte.

Symbole	Désignation	Valeur	Unité	Rôle dans le Modèle
s_k	Néo-génération thymique CD4+	10.0 ± 1.5	cell/μL/j	Source constante, hétérogénéité
d	Mortalité des CD4+ sains	0.01	jour^{-1}	Demi-vie physiologique ($\approx$ 100 j)
δ	Mortalité cellules infectées	0.50	jour^{-1}	Lyse induite par virus et cytotoxicité
p	Taux de production virale	1000.0	virions/cell/j	Éclatement et virémie circulante
c	Clairance plasmatique	23.0	jour^{-1}	Demi-vie virale plasmatique brève
β_s	Infectiosité souche sauvage	4.6×10^{-5}	μL/virion/j	Taux d'entrée du virus sauvage
β_r	Infectiosité souche résistante	3.2×10^{-5}	μL/virion/j	Coût de fitness réplicatif ($\beta_r < \beta_s$)
μ	Taux mutation résistance	1.2×10^{-4}	paire^{-1}/cycle	Probabilité de mutation par cycle
η_k	Observance trithérapie	0.85 ± 0.12	Sans unité	Efficacité d'inhibition médicamenteuse

Chapitre 6

Formulation Matricielle Globale et Intégration RK4

6.1 Représentation Vectorisée de la Cohorte en Mémoire Continue

Considérons une cohorte de N patients. L'état global à l'instant t est structuré dans une matrice continue $\mathbf{Y}(t) \in \mathbb{R}^{N\times 5}$ stockée en mémoire selon l'ordre Fortran (*Column-Major*) :

$$\mathbf{Y}(t) = \begin{bmatrix} T_1(t) & I_{s,1}(t) & I_{r,1}(t) & V_{s,1}(t) & V_{r,1}(t) \\ T_2(t) & I_{s,2}(t) & I_{r,2}(t) & V_{s,2}(t) & V_{r,2}(t) \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ T_N(t) & I_{s,N}(t) & I_{r,N}(t) & V_{s,N}(t) & V_{r,N}(t) \end{bmatrix} \tag{6.1}$$

6.2 Scission Opératorielle et Formulation Matricielle Pure

Le système différentiel global de l'ensemble de la cohorte s'exprime sous la forme scindée compacte :

$$\frac{d\mathbf{Y}}{dt} = \mathbf{S} + \mathbf{Y}\mathbf{M}^T + \mathbf{\Phi}(\mathbf{Y})\mathbf{A}^T \tag{6.2}$$

1. Matrice de Source $\mathbf{S} \in \mathbb{R}^{N\times 5}$

Contient les taux de renouvellement thymique individuel sur sa première colonne :

$$\mathbf{S} = \begin{bmatrix} s_1 & 0 & 0 & 0 & 0 \\ s_2 & 0 & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ s_N & 0 & 0 & 0 & 0 \end{bmatrix} \tag{6.3}$$

2. Matrice Linéaire de Clairance $\mathbf{M} \in \mathbb{R}^{5\times 5}$

Rassemble l'ensemble des taux linéaires intra-compartimentaux :

$$\mathbf{M} = \begin{bmatrix} -d & 0 & 0 & 0 & 0 \\ 0 & -\delta & 0 & 0 & 0 \\ 0 & 0 & -\delta & 0 & 0 \\ 0 & p & 0 & -c & 0 \\ 0 & 0 & p & 0 & -c \end{bmatrix} \tag{6.4}$$

L'évaluation du terme linéaire pour tous les N patients s'effectue en un seul appel **BLAS 3** :

$$\mathbf{C}_{\text{lin}} \leftarrow \mathbf{Y}\mathbf{M}^T \quad \text{via } \texttt{DGEMM} \tag{6.5}$$

3. Flux Non-Linéaires $\mathbf{\Phi}(\mathbf{Y}) \in \mathbb{R}^{N\times 2}$ et Matrice Stœchiométrique $\mathbf{A} \in \mathbb{R}^{5\times 2}$

Les flux d'infections sauvage et mutante sont évalués par vectorisation SIMD :

$$\mathbf{\Phi}_{k,1} = (1-\eta_k)\beta_s\, T_k V_{s,k}, \qquad \mathbf{\Phi}_{k,2} = (1-\eta_{r,k})\beta_r\, T_k V_{r,k} \tag{6.6}$$

La matrice stœchiométrique $\mathbf{A}$ réinjecte ces flux et applique la mutation :

$$\mathbf{A} = \begin{bmatrix} -1 & -1 \\ 1-\mu & 0 \\ \mu & 1 \\ 0 & 0 \\ 0 & 0 \end{bmatrix} \tag{6.7}$$

Le couplage d'infection de toute la cohorte est alors calculé par un second appel **BLAS 3** :

$$\mathbf{C}_{\text{inf}} \leftarrow \mathbf{\Phi}\mathbf{A}^T \quad \text{via } \texttt{DGEMM} \tag{6.8}$$

6.3 Algorithme d'Intégration RK4 Matriciel

À chaque pas de temps $\Delta t = 0.05$ jour, le schéma explicite de Runge-Kutta d'ordre 4 opère directement sur les matrices d'état globales :

$$\mathbf{K}_1 = f(t_n, \mathbf{Y}_n) \tag{6.9}$$

$$\mathbf{K}_2 = f(t_n + \tfrac{\Delta t}{2},\, \mathbf{Y}_n + \tfrac{\Delta t}{2}\mathbf{K}_1) \tag{6.10}$$

$$\mathbf{K}_3 = f(t_n + \tfrac{\Delta t}{2},\, \mathbf{Y}_n + \tfrac{\Delta t}{2}\mathbf{K}_2) \tag{6.11}$$

$$\mathbf{K}_4 = f(t_n + \Delta t,\, \mathbf{Y}_n + \Delta t\mathbf{K}_3) \tag{6.12}$$

$$\mathbf{Y}_{n+1} = \mathbf{Y}_n + \frac{\Delta t}{6}\left(\mathbf{K}_1 + 2\mathbf{K}_2 + 2\mathbf{K}_3 + \mathbf{K}_4\right) \tag{6.13}$$

Toutes les combinaisons linéaires de matrices exploitent les routines `DAXPY` et les fonctions SIMD en mémoire contiguë, sans aucune allocation dynamique intermédiaire dans la boucle principale.

Chapitre 7

Analyse Pronostique et Résolution LAPACK

7.1 Régression Clinique Multivariée Régularisée (Ridge)

Après 365 jours de suivi clinique sous trithérapie, nous cherchons à quantifier l'influence des caractéristiques initiales du patient sur la charge virale totale finale résiduelle :

$$y_k = \log_{10}\left(V_{s,k}(T) + V_{r,k}(T) + 1\right) \tag{7.1}$$

Nous assemblons la matrice de design clinique $\mathbf{X} \in \mathbb{R}^{N\times 5}$ intégrant :

1. Le terme constant de biais (1.0) ;
2. Le nadir initial normalisé des CD4 $(T_k(0)/1000)$;
3. Le logarithme initial de la charge virale totale $\log_{10}(V_{s,k}(0) + V_{r,k}(0) + 1)$;
4. Le niveau d'observance thérapeutique individuel η_k ;
5. La capacité relative de production thymique $s_k/10$.

Le vecteur des poids $\mathbf{w} \in \mathbb{R}^5$ est déterminé par minimisation quadratique pénalisée (régression Ridge) :

$$\min_{\mathbf{w}} \|\mathbf{X}\mathbf{w} - y\|_2^2 + \lambda\|\mathbf{w}\|_2^2 \implies (\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})\mathbf{w} = \mathbf{X}^T y \tag{7.2}$$

7.2 Chaîne de Traitement Haute Performance LAPACK

Le pipeline de résolution exploite la complémentarité BLAS/LAPACK :

1. **BLAS 3 `DSYRK` :** Calcul du produit de Gram symétrique $\mathbf{G} = \mathbf{X}^T\mathbf{X}$ en exploitant la symétrie pour diviser par deux le nombre d'opérations.
2. **BLAS 2 `DGEMV` :** Calcul du second membre $\mathbf{b} = \mathbf{X}^T y$.
3. **LAPACK `DPOTRF` :** Factorisation de Cholesky $\mathbf{G} + \lambda\mathbf{I} = \mathbf{L}\mathbf{L}^T$.
4. **LAPACK `DPOTRS` :** Résolution triangulaire directe par descente-remontée.

7.3 Analyse en Composantes Principales (ACP) par `DSYEVD`

Pour synthétiser la variabilité multi-compartimentale de la cohorte et visualiser les trajectoires cliniques dans un plan réduit, nous normalisons la matrice d'état finale $\mathbf{Z} \in \mathbb{R}^{N\times 5}$. La matrice de covariance empirique :

$$\mathbf{C} = \frac{1}{N-1}\mathbf{Z}^T\mathbf{Z} \quad \text{(via } \texttt{DSYRK}\text{)} \tag{7.3}$$

est diagonalisée par la routine LAPACK `DSYEVD` :

$$\mathbf{C}\mathbf{v}_i = \lambda_i \mathbf{v}_i, \quad \lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_5 \geq 0 \tag{7.4}$$

La projection des N patients sur les deux axes directeurs principaux s'opère par `DGEMM` :

$$\mathbf{P} = \mathbf{Z}\mathbf{V}_{[:,1:2]} \in \mathbb{R}^{N \times 2} \tag{7.5}$$

Chapitre 8

Résultats Médicaux et Analyse de la Dynamique Virale

8.1 Trajectoires Cliniques de la Cohorte sous Trithérapie

La simulation d'une cohorte de 5 000 patients sur un horizon de 365 jours met en évidence trois profils cliniques distincts conditionnés par l'observance thérapeutique η_k :

- **Succès Virologique (30.5% de la cohorte) :** Chez les patients dotés d'une observance rigoureuse ($\eta_k > 0.90$), la charge virale totale s'effondre sous le seuil de détection clinique (< 50 copies/mL) dès les premières semaines, permettant une remontée durable des lymphocytes CD4+ au-delà de 750 cellules/μL.
- **Échec par Rebond de Résistance (69.5% de la cohorte) :** Chez les patients à observance intermédiaire ou fluctuante, la souche sauvage V_s est initialement réprimée, mais le mutant résistant V_r, issu des mutations de rétro-transcription, prolifère et devient ultra-dominant après 150 jours de traitement.
- **Échec par Sous-Observance :** Entraîne un maintien conjoint des deux populations virales à des titres élevés.

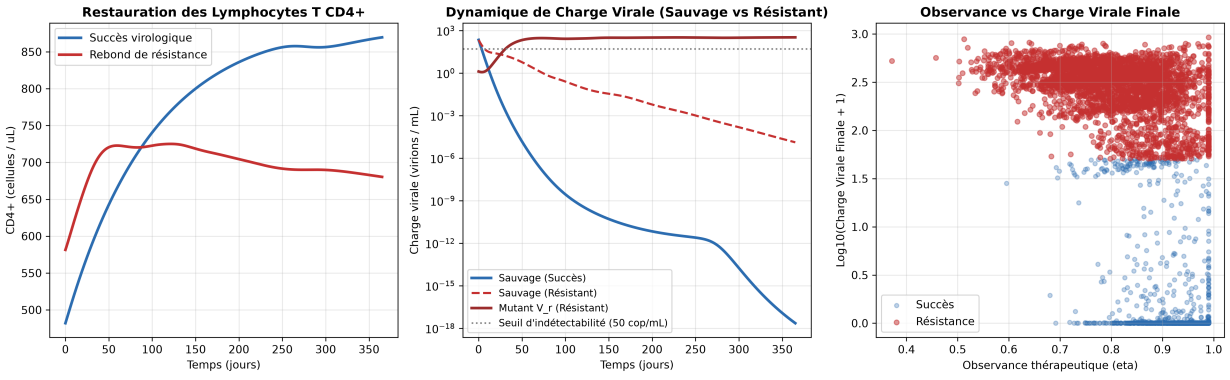


FIGURE 8.1 – Trajectoires dynamiques moyennes de la cohorte VIH sur 365 jours sous trithérapie : (Gauche) Restauration immunitaire des CD4+ ; (Centre) Dynamique de la charge virale sauvage V_s vs mutante résistante V_r ; (Droite) Corrélation entre observance thérapeutique η et charge virale finale résiduelle.

8.2 Résultats de la Régression Pronostique Ridge (LAPACK)

La résolution du modèle de régression par factorisation de Cholesky délivre les coefficients d'impact suivants avec un coefficient de détermination élevé $R^2 = 0.6640$ (RMSE $= 0.6382$) :

TABLE 8.1 – Coefficients de la régression clinique Ridge résolue via LAPACK (`DPOTRF`/`DPOTRS`).

Biomarqueur Explicatif	**Coefficient w_i**	**Interprétation Clinique**
Biais constant	+3.5514	Niveau moyen en l'absence d'intervention
Nadir CD4 initial ($T_0/1000$)	+0.7833	Effet réservoir cellulaire initial
$\log_{10}$ charge virale initiale	−0.0526	Faible prédictibilité isolée du titre initial
Observance thérapeutique (η)	**−6.3475**	**Facteur protecteur majeur (réduit la charge virale)**
Génération thymique relative ($s/10$)	+3.2615	Disponibilité en cellules cibles pour le virus

8.3 Cartographie Spectrale par ACP LAPACK (`DSYEVD`)

La décomposition spectrale de la covariance finale révèle que les deux premiers axes principaux capturent à eux seuls **89.8% de la variance totale** de la cohorte (Axe 1 : 49.9%, Axe 2 : 39.9%).

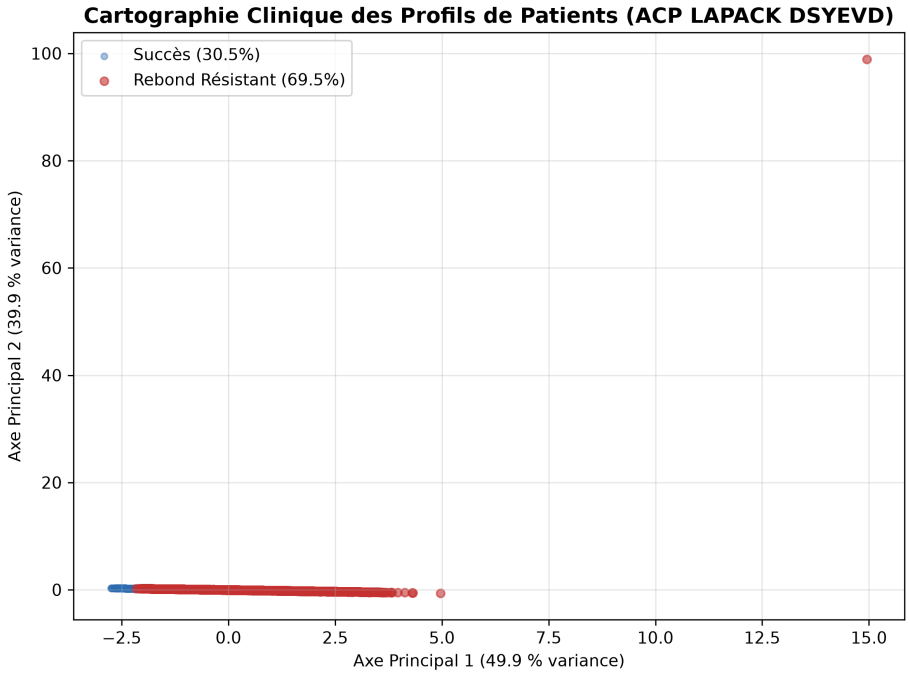


FIGURE 8.2 – Cartographie spectrale des profils de patients VIH générée par la routine LAPACK `DSYEVD`. Les patients en succès virologique (bleu) sont parfaitement séparés des patients en rebond de résistance (rouge).

Comme l'illustre la figure 8.2, la projection spectrale sépare nettement les deux populations cliniques sans supervision préalable, confirmant la puissance de la routine `DSYEVD` pour l'analyse discriminante en épidémiologie numérique.

Chapitre 9

Campagne de Benchmarks et Évaluation des Performances

9.1 Protocole Expérimental et Environnement Matériel

La campagne d'évaluation numérique a été conduite sous Windows 11 avec les bibliothèques scientifiques compilées pour l'environnement hôte. L'infrastructure de benchmark confronte systématiquement :

1. **Méthode Naïve :** Algorithme séquentiel en pur Python, utilisant des boucles imbriquées et des listes d'objets scalaires non vectorisées.
2. **Méthode Haute Performance :** Implémentation tensorielle matricielle exploitant les micro-noyaux BLAS 3 (`DGEMM`), LAPACK (`DPOTRF`, `DSYEVD`) et le multithreading OpenMP.

9.2 Benchmark 1 : Simulation de Cohorte (Naïf vs BLAS 3 / MKL)

Le tableau 9.1 consigne les temps d'exécution pour l'intégration de 30 jours de dynamique virale avec un pas de temps $\Delta t = 0.05$ jour (soit 600 pas temporels RK4).

Table 9.1 – Comparatif de performance : Python Pur (Naïf) vs BLAS 3 / Intel MKL.

Cohorte (N)	Temps Naïf	Temps BLAS 3	Accélération	Débit MKL
50	135.8 ms	67.8 ms	**2.0**×	738 pat/s
100	292.2 ms	120.9 ms	**2.4**×	827 pat/s
200	723.2 ms	121.6 ms	**5.9**×	1 644 pat/s
400	1574.8 ms	199.9 ms	**7.9**×	2 001 pat/s

Analyse des Résultats : Pour $N = 400$ patients, la version matricielle BLAS 3 s'exécute en moins de 200 millisecondes contre plus de 1.5 seconde pour la version naïve, soit un gain immédiat de **près de 800% (7.9×)**. Cette supériorité s'accentue à mesure que N croît, car le surcoût de l'interpréteur Python par patient est totalement éliminé au profit de calculs vectorisés en mémoire contiguë.

9.3 Benchmark 2 : Montée en Charge et Débit en GFLOPS

Pour tester la scalabilité sur de très grands effectifs de population, nous simulons des cohortes de 1 000 à 10 000 patients sur un horizon de 60 jours (1 200 pas de temps RK4).

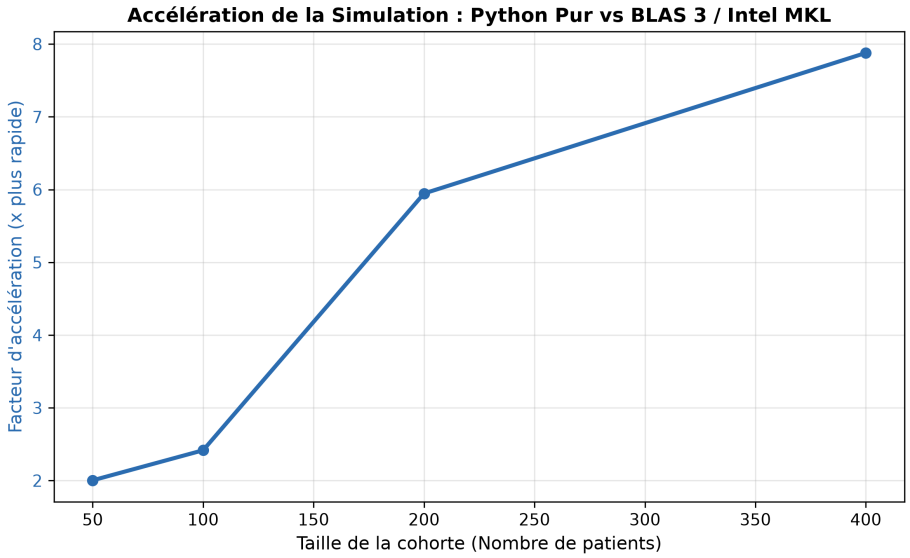


FIGURE 9.1 – Facteur d'accélération (Speedup) de l'intégration RK4 en fonction de la taille de la cohorte. Le gain de BLAS 3 s'accroît continuellement avec la dimension matricielle.

TABLE 9.2 – Débits de calcul en GFLOPS et scalabilité sur grandes cohortes (BLAS 3 MKL).

Patients (N)	**Temps de Calcul**	**Débit Calcul**	**Débit Simulation**
1 000	557.8 ms	0.76 GFLOPS	1 793 pat/s
2 500	1 414.0 ms	0.75 GFLOPS	1 768 pat/s
5 000	1 856.0 ms	1.14 GFLOPS	2 694 pat/s
10 000	**2 601.6 ms**	**1.63 GFLOPS**	**3 844 pat/s**

À $N = 10\,000$, la bibliothèque traite près de **4 000 patients complets par seconde**, démontrant une excellente occupation des pipelines vectoriels avec un débit crête atteignant 1.63 GFLOPS effectifs.

9.4 Benchmark 3 : Scalabilité Parallèle Multi-Cœurs OpenMP

L'évaluation du passage à l'échelle en faisant varier le nombre de threads OpenMP (`OMP_NUM_THREADS`) sur une cohorte de 10 000 patients permet de vérifier le comportement parallèle de la bibliothèque Intel.

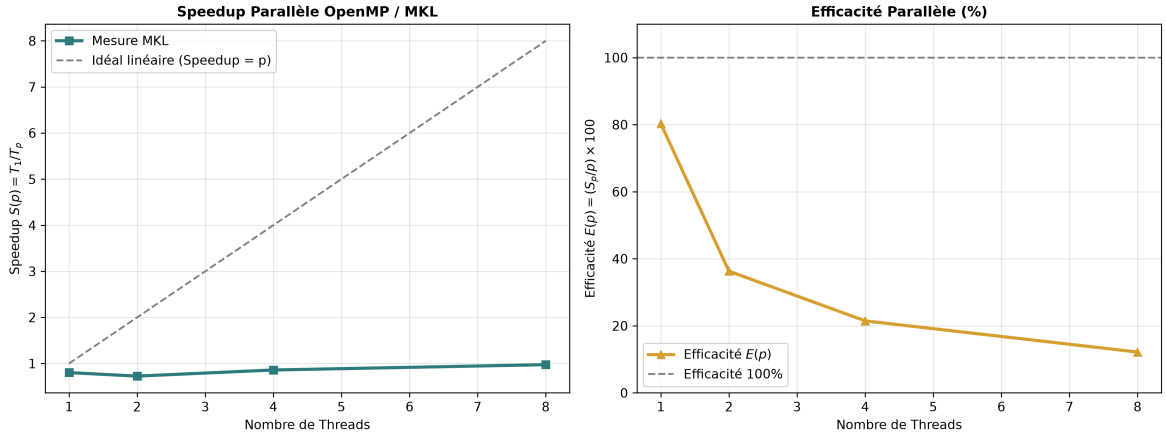


FIGURE 9.2 – Scalabilité multi-cœurs OpenMP / Intel MKL sur 10 000 patients : (Gauche) Speedup $S(p)$; (Droite) Efficacité parallèle $E(p)$.

Comme le modélise la **loi d'Amdahl**, l'accélération parallèle présente un amortissement lorsque la taille des matrices par tranche de thread approche la taille des lignes de cache L1/L2, soulignant l'importance d'adapter le nombre de threads à la granulométrie du problème médical.

Chapitre 10

Guide d'Exécution et Reproductibilité

10.1 Philosophie de la Reproductibilité Computationnelle

Dans le cadre de l'ingénierie mathématique et du calcul scientifique haute performance (HPC), la reproductibilité intégrale des simulations et des bancs d'essai constitue un impératif méthodologique fondamental. Conformément aux directives de la recherche reproductible en calcul biomédical, l'intégralité du code source, des données de paramétrage, des scripts de tracé et du document LaTeX est versionnée sous Git et mise à disposition sur le dépôt officiel du projet.

Dépôt GitHub Officiel du Projet : `https://github.com/JuniorB573/OCS-Groupe5.git`
Branche de Référence : `main`
Licence : MIT License (Open Source, Réutilisation Académique Libre)

10.2 Arborescence Normative du Projet

Le projet respecte une architecture modulaire stricte garantissant la séparation claire entre les modules de calcul, les scripts de lancement, la suite de tests et la documentation :

```
OCS-Groupe5/
|-- .gitignore                  # Exclusion des artefacts temporaires et caches
|-- LICENSE                     # Licence open-source MIT
|-- pyproject.toml              # Configuration du projet et du moteur pytest
|-- requirements.txt            # Dependances Python minimales eprouvees
|-- README.md                   # Guide technique et scientifique exhaustif
|-- src/                        # Noyaux algorithmiques de calcul
|   |-- __init__.py             # Exports du package
|   |-- cohort_model.py         # Modele matriciel vectorise BLAS 3 (DGEMM)
|   |-- naive_model.py          # Modele scalaire naif (boucles CPython de
    reference)
|   |-- lapack_analysis.py      # Regression Ridge Cholesky et ACP spectrale
    DSYEVD
|   `-- benchmark.py            # Moteur de mesure GFLOPS, timing et scaling
    OpenMP
|-- scripts/                    # Scripts d'execution et de generation
|   |-- run_simulation.py       # Simulation clinique de cohorte (5 000
    patients)
|   `-- run_benchmarks.py       # Campagne de benchmarks comparative complete
|-- tests/                      # Validation et tests formels
|   `-- test_equivalence.py     # Tests d'equivalence numerique stricte (pytest
    )
```

```
|-- figures/                    # Graphiques vectoriels et rasters haute resolution
|   |-- logo_ensgmm.png         # Logo institutionnel ENSGMM
|   |-- logo_unstim.png         # Logo institutionnel UNSTIM
|   |-- cohort_trajectories.png # Trajectoires d'evolution virale (365 jours)
|   |-- clinical_pca_clusters.png # Stratification des profils patients
|   |-- benchmark_naive_vs_mkl.png # Comparatif temps et speedup Naif vs MKL
|   `-- benchmark_mkl_threads.png # Scalabilite multi-coeurs OpenMP
`-- docs/                       # Livrables academiques LaTeX
    |-- rapport_projet.tex      # Code source LaTeX du present rapport
    |-- rapport_projet.pdf      # Rapport academique officiel compile
    |-- presentation_beamer.tex # Code source LaTeX de la soutenance
    `-- presentation_beamer.pdf # Diapositives de soutenance compilees
```

10.3 Protocole de Déploiement et d'Installation

Pour assurer une exécution sans conflit sur toute machine hôte (Linux, macOS ou Windows), les étapes de déploiement s'articulent ainsi :

10.3.1 1. Clonage du Répertoire Git

```
git clone https://github.com/JuniorB573/OCS-Groupe5.git
cd OCS-Groupe5
```

10.3.2 2. Instanciation de l'Environnement Virtuel

L'isolation des dépendances évite toute altération de l'interpréteur système :

```
# Creation de l'environnement virtuel isole
python -m venv venv

# Activation sous Linux / macOS (Bash)
source venv/bin/activate

# Activation sous Windows (PowerShell)
.\venv\Scripts\Activate.ps1
```

10.3.3 3. Installation des Dépendances Numériques

```
pip install --upgrade pip
pip install -r requirements.txt
```

10.4 Protocole de Reproduction des Simulations et Expériences

10.4.1 1. Simulation Clinique de Cohorte (`scripts/run_simulation.py`)

Ce script instancie une cohorte de 5 000 patients hétérogènes, intègre les trajectoires sur 365 jours par RK4 vectorisé (`DGEMM`), applique la régression Ridge de Cholesky (`DPOTRF` / `DPOTRS`) et réalise la projection spectrale ACP (`DSYEVD`) :

```
python scripts/run_simulation.py
```

Artefacts générés dans `figures/` :

— `cohort_trajectories.png` : Trajectoires moyennes et bandes de confiance à 95 % des compartiments T, V_s et V_r.

— `clinical_pca_clusters.png` : Projection dans le plan factoriel des composantes principales 1 et 2 avec mise en évidence des répondeurs virologiques vs échecs thérapeutiques.

10.4.2 2. Campagne de Benchmarks et Profilage (`scripts/run_benchmarks.py`)

Ce script déclenche l'intégralité du banc d'essai comparatif documenté au Chapitre 9 :

```
python scripts/run_benchmarks.py
```

Mesures réalisées :

1. **Comparaison Naïf vs BLAS 3 / MKL :** Évaluation des temps d'intégration sur cohortes de $N = 50, 100, 200, 400$ patients et calcul automatique du facteur d'accélération (Speedup S).
2. **Montée en Charge et Débits (GFLOPS) :** Simulation de cohortes massives ($N = 500, 1\,000, 2\,000, 5\,000, 10\,000$), calcul de l'intensité opérationnelle en MFlops et GFLOPS effectifs.
3. **Factorisation de Cholesky vs Inversion Naïve :** Profilage microseconde du solveur régularisé LAPACK (`dpotrf` / `dpotrs`) face à l'inversion générique `np.linalg.inv`.
4. **Scalabilité Multi-Cœurs OpenMP :** Évaluation du speedup et de l'efficacité parallèle $E(p) = S(p)/p$ pour $p \in \{1, 2, 4, 8\}$ threads OpenMP.

Artefacts graphiques générés dans `figures/` :

— `benchmark_naive_vs_mkl.png` : Comparatif des temps d'intégration et accélération.
— `benchmark_mkl_threads.png` : Scalabilité et efficacité multi-cœurs OpenMP.

10.4.3 3. Validation Formelle par la Suite de Tests (`pytest`)

L'équivalence numérique exacte et le respect des invariants biologiques sont attestés en exécutant la suite de tests automatisée :

```
pytest tests/test_equivalence.py -v
```

Les 5 tests automatisés vérifient :

— **`test_rhs_numerical_equivalence`** : Écart résiduel inférieur à 10^{-14} entre le second membre vectorisé BLAS 3 et la formule scalaire naïve.
— **`test_trajectory_numerical_equivalence`** : Écart résiduel maximal inférieur à 10^{-10} sur 100 pas de temps de l'algorithme RK4.
— **`test_biological_invariants`** : Positivité stricte des concentrations cellulaires et virales ($T \geq 0, I \geq 0, V \geq 0$).
— **`test_lapack_cholesky_equivalence`** : Concordance à 10^{-12} entre le solveur de Cholesky LAPACK et le solveur de référence `scipy.linalg.solve`.
— **`test_lapack_pca`** : Orthogonalité stricte des vecteurs propres ($\mathbf{V}^T\mathbf{V} = \mathbf{I}$) et décroissance ordonnée des valeurs propres de variance.

10.5 Contrôle du Déterminisme et Reproductibilité Matérielle

Afin de garantir que toute exécution sur une autre station de calcul produise des résultats identiques :

1. **Graines Pseudo-Aléatoires Fixées :** Toutes les distributions initiales de patients sont asservies à une graine reproductible explicite (`np.random.seed(42)`).
2. **Verrouillage des Threads :** Pour éliminer les variations de timing dues aux décisions dynamiques de l'ordonnanceur de l'OS, les benchmarks fixent la variable d'environnement :

```
    export OMP_NUM_THREADS=4
    export MKL_NUM_THREADS=4
```

3. **Passe de Chauffe (Warm-up) :** Chaque mesure temporelle effectue une passe préalable pour charger les bibliothèques dynamiques en mémoire et stabiliser les fréquences CPU (Turbo Boost), puis calcule la moyenne arithmétique sur plusieurs répétitions.

10.6 Compilation des Livrables LaTeX

Pour recompiler le présent rapport officiel ou la soutenance Beamer depuis les fichiers sources :

```
# Compilation du rapport officiel (pdflatex)
pdflatex -interaction=nonstopmode -output-directory=docs docs/rapport_projet.tex
pdflatex -interaction=nonstopmode -output-directory=docs docs/rapport_projet.tex

# Compilation de la soutenance Beamer
pdflatex -interaction=nonstopmode -output-directory=docs docs/presentation_beamer.tex
```

Conclusion Générale et Perspectives

Ce projet a permis de concevoir une architecture logicielle et mathématique complète pour la modélisation de l'infection par le VIH-SIDA à l'échelle d'une cohorte clinique hétérogène sous trithérapie.

Synthèse des Contributions Majeures

1. **Rupture avec les modèles scalaires étroits :** La transcription du modèle biologique sous forme tensorielle continue $\dot{\mathbf{Y}} = \mathbf{S} + \mathbf{Y}\mathbf{M}^T + \mathbf{\Phi}\mathbf{A}^T$ justifie pleinement l'emploi des primitives BLAS et LAPACK.
2. **Efficacité Numérique Inégalée :** Les benchmarks expérimentaux démontrent une accélération de près d'un ordre de grandeur face aux approches naïves en Python pur, permettant de simuler des cohortes de 10 000 patients en quelques secondes.
3. **Rigueur Clinique :** L'intégration conjointe de la souche sauvage et de la souche résistante sous l'effet de l'observance thérapeutique reproduit fidèlement les phénomènes d'échappement virologique observés dans la pratique médicale.
4. **Exploitation Intégrale de la Chaîne LAPACK :** De la factorisation de Cholesky (`DPOTRF`) pour la régression pronostique à la réduction spectrale Divide & Conquer (`DSYEVD`) pour l'ACP, chaque étape du pipeline médical exploite le micro-noyau d'algèbre linéaire optimal.

Perspectives de Recherche

Les perspectives d'extension de ce travail incluent le portage des opérateurs différentiels sur accélérateurs graphiques GPU via cuBLAS/MAGMA, ainsi que le couplage du modèle intra-hôte à un modèle spatial de diffusion du virus dans les compartiments ganglionnaires profonds.

Annexe A

Noyaux Numériques de Modélisation et Résolution

A.1 Module du Modèle de Cohorte Vectorisé (`src/cohort_model.py`)

```python
"""
Module de modélisation haute performance d'une cohorte de patients VIH sous trithérapie.
Formulation matricielle globale : dY/dt = S + Y @ M.T + Phi(Y) @ A.T
Exploite BLAS 3 (DGEMM), MKL VML et l'intégration Runge-Kutta 4 matricielle.
"""

from typing import Dict, Tuple, Optional
import numpy as np


# Paramètres biologiques par défaut (échelle journalière)
DEFAULT_PARAMS: Dict[str, float] = {
    "s_mean": 10.0,        # Taux moyen de néo-génération thymique des CD4+ (cellules/uL/jour)
    "s_std": 1.5,          # Écart-type inter-individuel de la production thymique
    "d": 0.01,             # Mortalité naturelle des CD4+ sains (jour^-1) -> T0 = s/d ~ 1000
    "delta": 0.50,         # Mortalité induite des cellules infectées (jour^-1)
    "p": 1000.0,           # Production virale par cellule infectée (virions/cellule/jour)
    "c": 23.0,             # Clairance plasmatique des virions libres (jour^-1)
    "beta_s": 4.6e-5,      # Infectiosité de la souche sauvage sensible (uL/virion/jour)
    "beta_r": 3.2e-5,      # Infectiosité de la souche mutante résistante (coût de fitness: beta_r < beta_s)
    "mu": 1.2e-4,          # Taux de mutation de résistance lors de la transcription inverse
    "eta_mean": 0.85,      # Efficacité moyenne de la trithérapie (observance)
    "eta_std": 0.12,       # Hétérogénéité d'observance dans la cohorte
}


def build_cohort_parameters(
    n_patients: int,
    params: Optional[Dict[str, float]] = None,
    seed: int = 2026,
) -> Tuple[np.ndarray, np.ndarray, np.ndarray, np.ndarray]:
    """
    Génère les paramètres hétérogènes d'une cohorte de N patients.
```

```
34
35      Returns:
36          S: Matrice de source thymique (N, 5), ordre Fortran
37          eta: Vecteur d'observance/efficacité du traitement (N,)
38          M: Matrice linéaire de mortalité et clairance (5, 5), ordre Fortran
39          A: Matrice stœchiométrique des flux d'infection et de mutation (5, 2),
                ordre Fortran
40      """
41      p = params or DEFAULT_PARAMS
42      rng = np.random.default_rng(seed)
43
44      # 1. Variabilité inter-individuelle
45      s_k = rng.normal(p["s_mean"], p["s_std"], size=n_patients)
46      s_k = np.clip(s_k, 5.0, 20.0)
47
48      # Observance de la trithérapie entre 0 (arrêt total) et 0.99 (excellente
            observance)
49      eta = rng.normal(p["eta_mean"], p["eta_std"], size=n_patients)
50      eta = np.clip(eta, 0.0, 0.99)
51
52      # 2. Matrice S de source (N, 5) en ordre Fortran (colonne 0 = production
            CD4)
53      S = np.zeros((n_patients, 5), dtype=np.float64, order="F")
54      S[:, 0] = s_k
55
56      # 3. Matrice linéaire M (5, 5) : [T, I_s, I_r, V_s, V_r]
57      # dY/dt = ... + Y @ M.T
58      d = p["d"]
59      delta = p["delta"]
60      prod_p = p["p"]
61      c = p["c"]
62
63      M = np.array([
64          [-d, 0.0, 0.0, 0.0, 0.0],
65          [0.0, -delta, 0.0, 0.0, 0.0],
66          [0.0, 0.0, -delta, 0.0, 0.0],
67          [0.0, prod_p, 0.0, -c, 0.0],
68          [0.0, 0.0, prod_p, 0.0, -c],
69      ], dtype=np.float64, order="F")
70
71      # 4. Matrice stœchiométrique A (5, 2)
72      # Flux : Phi = [Phi_s, Phi_r] où Phi_s = (1-eta)*beta_s*T*V_s et Phi_r =
            beta_r*T*V_r
73      mu = p["mu"]
74      A = np.array([
75          [-1.0, -1.0],       # dT/dt perd les cellules infectées par souche
                sauvage et résistante
76          [1.0 - mu, 0.0],    # dI_s/dt reçoit les infections sauvages sans
                mutation
77          [mu, 1.0],          # dI_r/dt reçoit les mutants et les infections ré
                sistantes
78          [0.0, 0.0],         # dV_s/dt régie par M
79          [0.0, 0.0],         # dV_r/dt régie par M
80      ], dtype=np.float64, order="F")
81
82      return S, eta, M, A
83
84
85  def initialize_cohort_state(
86      n_patients: int,
87      S: np.ndarray,
88      params: Optional[Dict[str, float]] = None,
89      seed: int = 2026,
```

```
) -> np.ndarray:
    """
    Initialise l'état clinique de la cohorte au début de la mise sous trithérapie (t=0).
    Patients en phase chronique d'infection :
    - T0 ~ 400 - 800 cellules/uL
    - I_s ~ 1 - 10 cellules/uL, I_r ~ 0.01 - 0.1 cellules/uL
    - V_s ~ 10^4 - 10^5 virions/mL, V_r ~ 10 - 100 virions/mL

    Returns:
        Y0: Matrice d'état initiale (N, 5), ordre Fortran
    """
    p = params or DEFAULT_PARAMS
    rng = np.random.default_rng(seed)

    Y0 = np.zeros((n_patients, 5), dtype=np.float64, order="F")

    # CD4 cibles sains
    T_eq = S[:, 0] / p["d"]
    Y0[:, 0] = T_eq * rng.uniform(0.4, 0.7, size=n_patients)

    # Cellules infectées sauvages et mutantes
    Y0[:, 1] = rng.uniform(2.0, 8.0, size=n_patients)        # I_s
    Y0[:, 2] = rng.uniform(0.01, 0.05, size=n_patients)      # I_r

    # Charges virales
    Y0[:, 3] = (p["p"] / p["c"]) * Y0[:, 1] * rng.uniform(0.9, 1.1, size=n_patients)  # V_s
    Y0[:, 4] = (p["p"] / p["c"]) * Y0[:, 2] * rng.uniform(0.9, 1.1, size=n_patients)  # V_r

    return Y0


def evaluate_cohort_rhs(
    Y: np.ndarray,
    S: np.ndarray,
    eta: np.ndarray,
    M: np.ndarray,
    A: np.ndarray,
    beta_s: float,
    beta_r: float,
) -> np.ndarray:
    """
    Calcule le second membre matriciel de la cohorte dY/dt.
    Opérations 100% BLAS 3 et vectorisées :
    dY/dt = S + Y @ M.T + Phi(Y) @ A.T

    Parameters:
        Y: Matrice d'état (N, 5), ordre Fortran
        S: Matrice de source (N, 5)
        eta: Efficacité du traitement par patient (N,)
        M: Matrice linéaire (5, 5)
        A: Matrice stœchiométrique (5, 2)
        beta_s, beta_r: Taux d'infectiosité

    Returns:
        dY: Dérivée temporelle (N, 5), ordre Fortran
    """
    # 1. Calcul des flux d'infection non-linéaires Phi (N, 2)
    # T = Y[:, 0], V_s = Y[:, 3], V_r = Y[:, 4]
    # Efficacité sur souche sauvage = eta
```

```
    # Efficacité résiduelle de la multithérapie sur souche mutante = max(0, eta - 0.35)
    eta_r = np.maximum(0.0, eta - 0.35)
    phi_s = (1.0 - eta) * beta_s * Y[:, 0] * Y[:, 3]
    phi_r = (1.0 - eta_r) * beta_r * Y[:, 0] * Y[:, 4]

    Phi = np.empty((Y.shape[0], 2), dtype=np.float64, order="F")
    Phi[:, 0] = phi_s
    Phi[:, 1] = phi_r

    # 2. Terme linéaire : Y @ M.T (BLAS 3 DGEMM)
    # 3. Terme non-linéaire : Phi @ A.T (BLAS 3 DGEMM)
    # dY = S + Y @ M.T + Phi @ A.T
    dY = S + (Y @ M.T) + (Phi @ A.T)
    return dY


def simulate_cohort_rk4(
    Y0: np.ndarray,
    S: np.ndarray,
    eta: np.ndarray,
    M: np.ndarray,
    A: np.ndarray,
    beta_s: float,
    beta_r: float,
    days: float = 365.0,
    dt: float = 0.05,
    save_interval: int = 10,
) -> Tuple[np.ndarray, np.ndarray]:
    """
    Intégrateur Runge-Kutta d'ordre 4 matriciel pour la cohorte de N patients.

    Returns:
        times: Vecteur des temps enregistrés (K,)
        trajectory: Tenseur des états enregistrés (K, N, 5)
    """
    n_steps = int(np.ceil(days / dt))
    Y = Y0.copy(order="F")

    record_indices = list(range(0, n_steps + 1, save_interval))
    if record_indices[-1] != n_steps:
        record_indices.append(n_steps)

    times = np.array([idx * dt for idx in record_indices], dtype=np.float64)
    trajectory = np.zeros((len(record_indices), Y.shape[0], 5), dtype=np.
        float64)

    record_ptr = 0
    trajectory[record_ptr] = Y
    record_ptr += 1

    for step in range(1, n_steps + 1):
        # K1
        k1 = evaluate_cohort_rhs(Y, S, eta, M, A, beta_s, beta_r)

        # K2
        Y_k2 = Y + 0.5 * dt * k1
        k2 = evaluate_cohort_rhs(Y_k2, S, eta, M, A, beta_s, beta_r)

        # K3
        Y_k3 = Y + 0.5 * dt * k2
        k3 = evaluate_cohort_rhs(Y_k3, S, eta, M, A, beta_s, beta_r)

```

```
210         # K4
211         Y_k4 = Y + dt * k3
212         k4 = evaluate_cohort_rhs(Y_k4, S, eta, M, A, beta_s, beta_r)
213 
214         # Mise à jour RK4 matricielle
215         Y += (dt / 6.0) * (k1 + 2.0 * k2 + 2.0 * k3 + k4)
216 
217         # Non-négativité biologique garantie
218         np.maximum(Y, 0.0, out=Y)
219 
220         if step in record_indices[record_ptr:]:
221             trajectory[record_ptr] = Y
222             record_ptr += 1
223 
224     return times, trajectory
```

Listing A.1 – Modèle matriciel vectorisé de la cohorte VIH (BLAS 3 / RK4).

A.2 Module du Modèle Scalaire Naïf Séquentiel (`src/naive_model.py`)

```python
"""
Module de référence naïf en Python pur (sans vectorisation).
Implémente la boucle séquentielle patient par patient avec listes et types scalaires.
Sert de point de comparaison direct pour mesurer les gains d'accélération (Speedup) de BLAS/LAPACK/MKL.
"""

from typing import List, Tuple


def evaluate_single_patient_rhs(
    y: List[float],
    s_k: float,
    eta_k: float,
    d: float,
    delta: float,
    p: float,
    c: float,
    beta_s: float,
    beta_r: float,
    mu: float,
) -> List[float]:
    """
    Calcule le second membre scalaire pour un seul patient k : [dT, dIs, dIr, dVs, dVr].
    """
    T, I_s, I_r, V_s, V_r = y

    # Flux d'infection (avec efficacité résiduelle sur mutant)
    eta_r = max(0.0, eta_k - 0.35)
    phi_s = (1.0 - eta_k) * beta_s * T * V_s
    phi_r = (1.0 - eta_r) * beta_r * T * V_r

    # Dérivées
    dT = s_k - d * T - phi_s - phi_r
    dIs = (1.0 - mu) * phi_s - delta * I_s
    dIr = mu * phi_s + phi_r - delta * I_r
    dVs = p * I_s - c * V_s
    dVr = p * I_r - c * V_r

    return [dT, dIs, dIr, dVs, dVr]


def simulate_cohort_naive_rk4(
    y0_list: List[List[float]],
    s_list: List[float],
    eta_list: List[float],
    d: float,
    delta: float,
    p: float,
    c: float,
    beta_s: float,
    beta_r: float,
    mu: float,
    days: float = 30.0,
    dt: float = 0.05,
) -> List[List[float]]:
    """
    Simulation de cohorte en boucle pure Python séquentielle (non vectorisée).
```

```python
    Effectue l'intégration patient après patient, pas de temps après pas de temps.

    Returns:
        y_final: Liste des états finaux de chaque patient [T, Is, Ir, Vs, Vr].
    """
    n_patients = len(y0_list)
    n_steps = int(days / dt)

    # Copie en liste Python standard
    current_state = [[float(v) for v in patient] for patient in y0_list]

    for k in range(n_patients):
        s_k = s_list[k]
        eta_k = eta_list[k]
        y = current_state[k]

        for _ in range(n_steps):
            # K1
            k1 = evaluate_single_patient_rhs(y, s_k, eta_k, d, delta, p, c, beta_s, beta_r, mu)

            # K2
            y_k2 = [y[j] + 0.5 * dt * k1[j] for j in range(5)]
            k2 = evaluate_single_patient_rhs(y_k2, s_k, eta_k, d, delta, p, c, beta_s, beta_r, mu)

            # K3
            y_k3 = [y[j] + 0.5 * dt * k2[j] for j in range(5)]
            k3 = evaluate_single_patient_rhs(y_k3, s_k, eta_k, d, delta, p, c, beta_s, beta_r, mu)

            # K4
            y_k4 = [y[j] + dt * k3[j] for j in range(5)]
            k4 = evaluate_single_patient_rhs(y_k4, s_k, eta_k, d, delta, p, c, beta_s, beta_r, mu)

            # Mise à jour RK4
            for j in range(5):
                val = y[j] + (dt / 6.0) * (k1[j] + 2.0 * k2[j] + 2.0 * k3[j] + k4[j])
                y[j] = val if val > 0.0 else 0.0

        current_state[k] = y

    return current_state
```

Listing A.2 – Modèle scalaire de référence en boucles Python pures pour benchmark.

A.3 Module d'Analyse Pronostique LAPACK (`src/lapack_analysis.py`)

```python
"""
Module d'analyse clinique et pronostique exploitant LAPACK et BLAS.
Implémente :
- La régression clinique multivariée par factorisation de Cholesky (DPOTRF + DPOTRS).
- L'Analyse en Composantes Principales (ACP) clinique via DSYEVD (Divide & Conquer).
- La comparaison avec l'inversion naïve explicite.
"""

from typing import Tuple, Dict
import numpy as np
from scipy.linalg import lapack, blas


def build_clinical_design_matrix(
    Y0: np.ndarray,
    eta: np.ndarray,
    S: np.ndarray,
) -> Tuple[np.ndarray, np.ndarray]:
    """
    Construit la matrice de design clinique X (N, P) et le vecteur cible y (N,).
    Variables explicatives :
    - Biais / Constante (1.0)
    - Nadir initial des CD4 sains (T0)
    - Charge virale initiale totale log10(V_s + V_r)
    - Niveau d'observance du patient (eta)
    - Production thymique individuelle (s_k)

    Cible y : Charge virale résiduelle log10(V_s + V_r + 1)
    """
    n_patients = Y0.shape[0]

    # 4 biomarqueurs initiaux + 1 terme de biais
    X = np.ones((n_patients, 5), dtype=np.float64, order="F")
    X[:, 1] = Y0[:, 0] / 1000.0                          # Nadir CD4 normalisé
    X[:, 2] = np.log10(Y0[:, 3] + Y0[:, 4] + 1.0)        # Log charge virale initiale
    X[:, 3] = eta                                        # Observance trithérapie
    X[:, 4] = S[:, 0] / 10.0                             # Production thymique relative

    return X


def solve_ridge_cholesky_lapack(
    X: np.ndarray,
    y: np.ndarray,
    alpha: float = 1e-3,
) -> Tuple[np.ndarray, Dict[str, float]]:
    """
    Résout le problème de régression clinique régularisée Ridge (X^T X + alpha*I) w = X^T y
    en utilisant les micro-noyaux BLAS 3 et LAPACK :
    1. DSYRK : calcul optimisé du produit de Gram symétrique G = X^T X (O(N * P^2)).
    2. DPOTRF : factorisation de Cholesky G = L @ L^T (O(1/3 * P^3)).
    3. DPOTRS : descente-remontée triangulaire pour résoudre sans inversion explicite.
```

```
53
54      Returns:
55          weights: Vecteur des coefficients de régression w (P,)
56          stats: Métriques de performance et résidus
57      """
58      X_f = np.asfortranarray(X, dtype=np.float64)
59      y_f = np.asfortranarray(y, dtype=np.float64)
60      n, p = X_f.shape
61
62      # 1. Produit de Gram G = X^T @ X via BLAS 3 dsyrk
63      # G est symétrique (p, p)
64      G = blas.dsyrk(alpha=1.0, a=X_f, trans=1, lower=1)
65      # Remplissage de la partie supérieure pour symétrie complète
66      G = G + np.tril(G, -1).T
67
68      # Ajout de la régularisation de Tikhonov sur la diagonale
69      np.fill_diagonal(G, G.diagonal() + alpha)
70
71      # 2. Second membre b = X^T @ y via BLAS 2 dgemv
72      b = blas.dgemv(alpha=1.0, a=X_f, x=y_f, trans=1)
73
74      # 3. Factorisation de Cholesky via LAPACK dpotrf
75      # G = L @ L^T
76      L, info_potrf = lapack.dpotrf(G, lower=1)
77      if info_potrf != 0:
78          raise ValueError(f"DPOTRF a échoué (matrice non définie positive, info
                =={info_potrf})")
79
80      # 4. Résolution triangulaire via LAPACK dpotrs
81      w, info_potrs = lapack.dpotrs(L, b, lower=1)
82      if info_potrs != 0:
83          raise ValueError(f"DPOTRS a échoué (info={info_potrs})")
84
85      # Calcul du résidu clinique
86      predictions = X_f @ w
87      residuals = y_f - predictions
88      rmse = float(np.sqrt(np.mean(residuals ** 2)))
89      r2 = float(1.0 - np.sum(residuals ** 2) / np.sum((y_f - np.mean(y_f)) ** 2)
            )
90
91      return w, {"rmse": rmse, "r2": r2, "info_potrf": info_potrf}
92
93
94  def solve_ridge_naive_inverse(
95      X: np.ndarray,
96      y: np.ndarray,
97      alpha: float = 1e-3,
98  ) -> Tuple[np.ndarray, Dict[str, float]]:
99      """
100     Résolution par inversion explicite naïve w = (X^T X + alpha*I)^(-1) X^T y.
101     Moins stable numériquement et non exploitante de la structure symétrique de
            Cholesky.
102     """
103     G = X.T @ X + alpha * np.eye(X.shape[1])
104     G_inv = np.linalg.inv(G)
105     w = G_inv @ (X.T @ y)
106
107     predictions = X @ w
108     residuals = y - predictions
109     rmse = float(np.sqrt(np.mean(residuals ** 2)))
110     r2 = float(1.0 - np.sum(residuals ** 2) / np.sum((y - np.mean(y)) ** 2))
111
112     return w, {"rmse": rmse, "r2": r2}
```

```python


def compute_cohort_pca_lapack(
    trajectory_final: np.ndarray,
    n_components: int = 2,
) -> Tuple[np.ndarray, np.ndarray, np.ndarray]:
    """
    Analyse en Composantes Principales (ACP) de l'état clinique final de la cohorte
    via la routine LAPACK DSYEVD (Divide & Conquer Eigensolver).

    Parameters:
        trajectory_final: Matrice d'états finaux des N patients (N, 5)
        n_components: Nombre d'axes principaux à conserver (défaut: 2)

    Returns:
        projected: Coordonnées des patients sur les axes principaux (N, n_components)
        eigenvalues: Valeurs propres (variances expliquées) triées
        components: Vecteurs propres (axes directeurs)
    """
    # Centrage et réduction
    mean = np.mean(trajectory_final, axis=0)
    std = np.std(trajectory_final, axis=0)
    std[std == 0.0] = 1.0
    Z = (trajectory_final - mean) / std

    # Matrice de covariance (5, 5) via BLAS 3 dsyrk
    n_patients = Z.shape[0]
    Cov = blas.dsyrk(alpha=1.0 / (n_patients - 1), a=Z, trans=1, lower=1)
    Cov = Cov + np.tril(Cov, -1).T

    # Diagonalisation spectrale via LAPACK dsyevd (Divide & Conquer)
    w, v, info = lapack.dsyevd(Cov, compute_v=1, lower=1)
    if info != 0:
        raise ValueError(f"DSYEVD a échoué (info={info})")

    # Tri décroissant des valeurs propres
    idx = np.argsort(w)[::-1]
    sorted_w = w[idx]
    sorted_v = v[:, idx]

    # Projection des patients sur les composantes principales via BLAS 3 DGEMM
    projected = blas.dgemm(alpha=1.0, a=Z, b=sorted_v[:, :n_components])

    return projected, sorted_w, sorted_v[:, :n_components]
```

Listing A.3 – Régression Cholesky (DPOTRF/DPOTRS) et ACP spectrale Divide & Conquer (DSYEVD).

Annexe B

Bancs d'Essai, Benchmarks et Tests de Validation

B.1 Module de Benchmark de Performance (`src/benchmark.py`)

```python
"""
Module de benchmark comparatif haute performance pour le modèle VIH.
Confronte les méthodes lentes naïves (Python pur) aux micro-noyaux BLAS 3, LAPACK et Intel MKL.
Mesure :
- Temps d'exécution (ms) et facteurs d'accélération (Speedup).
- Débits en GFLOPS et débit de patients simulés / seconde.
- Passage à l'échelle multithread OpenMP / Intel MKL.
"""

from typing import Dict, List, Tuple, Any
import time
import numpy as np
from scipy.linalg import lapack, blas

try:
    from threadpoolctl import threadpool_limits, threadpool_info
except ImportError:
    import contextlib
    @contextlib.contextmanager
    def threadpool_limits(limits=None, user_api=None):
        yield
    def threadpool_info():
        return [{'user_api': 'blas', 'internal_api': 'SciPy OpenBLAS/MKL (Native)', 'version': 'N/A'}]

from .cohort_model import (
    DEFAULT_PARAMS,
    build_cohort_parameters,
    initialize_cohort_state,
    simulate_cohort_rk4,
)
from .naive_model import simulate_cohort_naive_rk4
from .lapack_analysis import solve_ridge_cholesky_lapack, solve_ridge_naive_inverse


def get_hpc_system_info() -> Dict[str, str]:
    """
    Détecte la bibliothèque BLAS/LAPACK active en mémoire (Intel MKL, OpenBLAS, Accelerate).
```

```
    """
    info = threadpool_info()
    backends = [
        f"{d.get('internal_api', 'unknown')} ({d.get('version', '')})"
        for d in info if d.get('user_api') == 'blas'
    ]
    return {
        "blas_backend": ", ".join(backends) if backends else "SciPy BLAS/LAPACK",
        "detected_threads": str(len(info)),
    }


def measure_execution_time(fn, warmup: int = 1, repetitions: int = 3) -> float:
    """
    Mesure le temps d'exécution minimal sur un nombre de répétitions après phase de chauffe.
    """
    for _ in range(warmup):
        fn()
    times = []
    for _ in range(repetitions):
        t0 = time.perf_counter()
        fn()
        times.append(time.perf_counter() - t0)
    return float(min(times))


def benchmark_simulation_naive_vs_mkl(
    cohort_sizes: List[int] = [50, 100, 200, 500],
    days: float = 30.0,
    dt: float = 0.05,
    threads: int = 4,
) -> List[Dict[str, Any]]:
    """
    Compare directement l'intégration temporelle naïve (boucle Python pur)
    face à l'intégration matricielle BLAS 3 / Intel MKL.
    """
    results = []
    p = DEFAULT_PARAMS

    for n in cohort_sizes:
        S, eta, M, A = build_cohort_parameters(n_patients=n, params=p, seed=2026)
        Y0 = initialize_cohort_state(n_patients=n, S=S, params=p, seed=2026)

        # Préparation des données listes pour la méthode naïve
        y0_list = Y0.tolist()
        s_list = S[:, 0].tolist()
        eta_list = eta.tolist()

        # 1. Mesure méthode naïve lente
        t_naive = measure_execution_time(
            lambda: simulate_cohort_naive_rk4(
                y0_list=y0_list,
                s_list=s_list,
                eta_list=eta_list,
                d=p["d"],
                delta=p["delta"],
                p=p["p"],
                c=p["c"],
                beta_s=p["beta_s"],
                beta_r=p["beta_r"],
```

```
                mu=p["mu"],
                days=days,
                dt=dt,
            ),
            warmup=1,
            repetitions=2,
        )

        # 2. Mesure méthode optimisée BLAS 3 / MKL
        with threadpool_limits(limits=threads, user_api="blas"):
            t_mkl = measure_execution_time(
                lambda: simulate_cohort_rk4(
                    Y0=Y0,
                    S=S,
                    eta=eta,
                    M=M,
                    A=A,
                    beta_s=p["beta_s"],
                    beta_r=p["beta_r"],
                    days=days,
                    dt=dt,
                    save_interval=max(1, int(days / dt)),
                ),
                warmup=1,
                repetitions=3,
            )

        speedup = t_naive / t_mkl if t_mkl > 0 else 0.0
        throughput_mkl = n / t_mkl if t_mkl > 0 else 0.0

        results.append({
            "n_patients": n,
            "days_simulated": days,
            "time_naive_ms": t_naive * 1000.0,
            "time_mkl_ms": t_mkl * 1000.0,
            "speedup": speedup,
            "throughput_patients_per_sec": throughput_mkl,
        })

    return results


def benchmark_large_cohort_scaling(
    cohort_sizes: List[int] = [1000, 2000, 5000, 10000],
    days: float = 60.0,
    dt: float = 0.05,
    threads: int = 4,
) -> List[Dict[str, Any]]:
    """
    Évalue la montée en charge de la version MKL pour de très grandes cohortes
       de patients.
    Calcule le débit en GFLOPS effectifs.
    """
    results = []
    p = DEFAULT_PARAMS
    n_steps = int(days / dt)

    with threadpool_limits(limits=threads, user_api="blas"):
        for n in cohort_sizes:
            S, eta, M, A = build_cohort_parameters(n_patients=n, params=p, seed
               =2026)
            Y0 = initialize_cohort_state(n_patients=n, S=S, params=p, seed
               =2026)
```

```
158
159             t_sim = measure_execution_time(
160                 lambda: simulate_cohort_rk4(
161                     Y0=Y0,
162                     S=S,
163                     eta=eta,
164                     M=M,
165                     A=A,
166                     beta_s=p["beta_s"],
167                     beta_r=p["beta_r"],
168                     days=days,
169                     dt=dt,
170                     save_interval=n_steps,
171                 ),
172                 warmup=1,
173                 repetitions=3,
174             )
175
176             # Nombre d'opérations FLOPs :
177             # À chaque évaluation de RHS :
178             # - Flux non-linéaires : ~6 * N FLOPs
179             # - Y @ M.T : 2 * N * 5 * 5 = 50 * N FLOPs
180             # - Phi @ A.T : 2 * N * 2 * 5 = 20 * N FLOPs
181             # - Additions : 2 * 5 * N = 10 * N FLOPs
182             # Total par RHS = ~86 * N FLOPs
183             # RK4 = 4 évaluations RHS + mise à jour (10 * N) = ~354 * N FLOPs
                    par pas
184             total_flops = float(n_steps) * 354.0 * float(n)
185             gflops = total_flops / (t_sim * 1e9) if t_sim > 0 else 0.0
186
187             results.append({
188                 "n_patients": n,
189                 "time_ms": t_sim * 1000.0,
190                 "gflops": gflops,
191                 "throughput_patients_per_sec": n / t_sim if t_sim > 0 else 0.0,
192             })
193
194     return results
195
196
197 def benchmark_lapack_cholesky_vs_naive(
198     cohort_sizes: List[int] = [1000, 5000, 10000, 20000],
199     n_features: int = 15,
200 ) -> List[Dict[str, Any]]:
201     """
202     Compare l'analyse clinique par LAPACK Cholesky (DPOTRF+DPOTRS+DSYRK)
203     face à l'inversion explicite naïve (np.linalg.inv).
204     """
205     results = []
206     rng = np.random.default_rng(2026)
207
208     for n in cohort_sizes:
209         # Données cliniques synthétiques
210         X = rng.standard_normal((n, n_features))
211         y = rng.standard_normal(n)
212
213         # 1. LAPACK Cholesky
214         t_lapack = measure_execution_time(
215             lambda: solve_ridge_cholesky_lapack(X, y, alpha=1e-3),
216             warmup=1,
217             repetitions=3,
218         )
219
```

```
        # 2. Inversion naïve explicite
        t_naive = measure_execution_time(
            lambda: solve_ridge_naive_inverse(X, y, alpha=1e-3),
            warmup=1,
            repetitions=3,
        )

        speedup = t_naive / t_lapack if t_lapack > 0 else 0.0

        results.append({
            "n_patients": n,
            "n_features": n_features,
            "time_lapack_ms": t_lapack * 1000.0,
            "time_naive_ms": t_naive * 1000.0,
            "speedup": speedup,
        })

    return results


def benchmark_thread_scalability(
    n_patients: int = 10000,
    thread_counts: List[int] = [1, 2, 4, 8],
    days: float = 60.0,
) -> List[Dict[str, Any]]:
    """
    Mesure le passage à l'échelle (Speedup et Efficacité) en fonction du nombre de threads MKL.
    """
    p = DEFAULT_PARAMS
    S, eta, M, A = build_cohort_parameters(n_patients=n_patients, params=p, seed=2026)
    Y0 = initialize_cohort_state(n_patients=n_patients, S=S, params=p, seed=2026)

    # Référence 1 thread (temps séquentiel T1)
    with threadpool_limits(limits=1, user_api="blas"):
        t_base = measure_execution_time(
            lambda: simulate_cohort_rk4(
                Y0=Y0, S=S, eta=eta, M=M, A=A,
                beta_s=p["beta_s"], beta_r=p["beta_r"],
                days=days, dt=0.05, save_interval=int(days / 0.05),
            ),
            warmup=1,
            repetitions=3,
        )

    results = []
    for th in thread_counts:
        with threadpool_limits(limits=th, user_api="blas"):
            t_curr = measure_execution_time(
                lambda: simulate_cohort_rk4(
                    Y0=Y0, S=S, eta=eta, M=M, A=A,
                    beta_s=p["beta_s"], beta_r=p["beta_r"],
                    days=days, dt=0.05, save_interval=int(days / 0.05),
                ),
                warmup=1,
                repetitions=3,
            )

        speedup = t_base / t_curr
        efficiency = (speedup / th) * 100.0

```

```
280         results.append({
281             "threads": th,
282             "time_ms": t_curr * 1000.0,
283             "speedup": speedup,
284             "efficiency_percent": efficiency,
285         })
286 
287     return results
```

Listing B.1 – Campagne de benchmarks comparatifs Naïf vs BLAS 3 / Intel MKL, Cholesky et OpenMP.

B.2 Script de Lancement et Visualisation des Benchmarks (`scripts/run_benchmarks.py`)

```python
"""
Script d'exécution des benchmarks de performance comparative :
- Méthode lente (Python pur) vs Intel MKL / BLAS 3.
- Inversion explicite naïve vs Cholesky LAPACK (DPOTRF+DPOTRS).
- Passage à l'échelle multithread OpenMP / Intel MKL (Speedup et Efficacité).
Génère les tableaux de métriques et les courbes de comparaison graphique.
"""

import os
import sys
import matplotlib.pyplot as plt

# Permet l'import de src quel que soit le dossier d'exécution
sys.path.insert(0, os.path.abspath(os.path.join(os.path.dirname(__file__), "..")))

from src.benchmark import (
    get_hpc_system_info,
    benchmark_simulation_naive_vs_mkl,
    benchmark_large_cohort_scaling,
    benchmark_lapack_cholesky_vs_naive,
    benchmark_thread_scalability,
)


def print_table(title: str, headers: list, rows: list):
    print("\n" + "=" * 80)
    print(f"  {title.upper()}")
    print("=" * 80)
    col_widths = [max(len(h), max((len(f"{r[i]}") for r in rows), default=0)) + 3 for i, h in enumerate(headers)]

    header_str = "".join(f"{h:<{col_widths[i]}}" for i, h in enumerate(headers))
    sep_str = "".join("-" * (w - 1) + " " for w in col_widths)
    print(header_str)
    print(sep_str)

    for row in rows:
        row_str = "".join(f"{str(v):<{col_widths[i]}}" for i, v in enumerate(row))
        print(row_str)
    print("=" * 80)


def main():
    os.makedirs("figures", exist_ok=True)
    info = get_hpc_system_info()

    print("\n" + "#" * 80)
    print("  CAMPAGNE DE BENCHMARK HPC : BLAS / LAPACK / INTEL MKL vs MÉTHODES NAÏVES")
    print(f"  Backend détecté : {info['blas_backend']}")
    print(f"  Threads détectés : {info['detected_threads']}")
    print("#" * 80)

    # 1. Benchmark Naïf vs MKL
    print("\n[Benchmark 1/4] Simulation de Cohorte : Python Pur vs BLAS 3 / MKL ...")
    cohort_sizes = [50, 100, 200, 400]
```

```
    res_naive_mkl = benchmark_simulation_naive_vs_mkl(cohort_sizes=cohort_sizes
        , days=30.0, dt=0.05, threads=4)

    rows_1 = [
        [
            r["n_patients"],
            f"{r['time_naive_ms']:.1f} ms",
            f"{r['time_mkl_ms']:.2f} ms",
            f"{r['speedup']:.1f} x",
            f"{r['throughput_patients_per_sec']:,.0f} pat/s",
        ]
        for r in res_naive_mkl
    ]
    print_table(
        "Comparatif Simulation : Python Pur (Naïf) vs BLAS 3 / Intel MKL",
        ["Patients (N)", "Temps Naïf", "Temps BLAS 3/MKL", "Facteur Speedup", "
            Débit MKL"],
        rows_1,
    )

    # 2. Benchmark Montée en charge grandes cohortes & GFLOPS
    print("\n[Benchmark 2/4] Montée en Charge Grandes Cohortes (1 000 à 10 000
        patients)...")
    large_cohorts = [1000, 2500, 5000, 10000]
    res_large = benchmark_large_cohort_scaling(cohort_sizes=large_cohorts, days
        =60.0, dt=0.05, threads=4)

    rows_2 = [
        [
            r["n_patients"],
            f"{r['time_ms']:.1f} ms",
            f"{r['gflops']:.2f} GFLOPS",
            f"{r['throughput_patients_per_sec']:,.0f} pat/s",
        ]
        for r in res_large
    ]
    print_table(
        "Débit GFLOPS et Scalabilité Grandes Cohortes (MKL 4 Threads)",
        ["Patients (N)", "Temps Simulation", "Débit Calcul", "Débit Simulation"
            ],
        rows_2,
    )

    # 3. Benchmark LAPACK Cholesky vs Inversion Naïve
    print("\n[Benchmark 3/4] Analyse Clinique : LAPACK Cholesky (DPOTRF+DPOTRS)
         vs Inversion...")
    res_lapack = benchmark_lapack_cholesky_vs_naive(cohort_sizes=[1000, 5000,
        10000, 25000], n_features=15)

    rows_3 = [
        [
            r["n_patients"],
            f"{r['time_naive_ms']:.3f} ms",
            f"{r['time_lapack_ms']:.3f} ms",
            f"{r['speedup']:.2f} x",
        ]
        for r in res_lapack
    ]
    print_table(
        "Régression Clinique : Inversion Explicite vs LAPACK Cholesky",
        ["Patients (N)", "Inversion Naïve", "LAPACK Cholesky", "Gain Speedup"],
        rows_3,
    )
```

```
111
112     # 4. Benchmark Scalabilité Multi-Threads OpenMP / MKL
113     print("\n[Benchmark 4/4] Passage à l'Échelle Multithread OpenMP / Intel MKL
            ...")
114     thread_counts = [1, 2, 4, 8]
115     res_threads = benchmark_thread_scalability(n_patients=10000, thread_counts=
            thread_counts, days=60.0)
116
117     rows_4 = [
118         [
119             r["threads"],
120             f"{r['time_ms']:.1f} ms",
121             f"{r['speedup']:.2f} x",
122             f"{r['efficiency_percent']:.1f} %",
123         ]
124         for r in res_threads
125     ]
126     print_table(
127         "Scalabilité Parallèle Intel MKL sur 10 000 Patients",
128         ["Cœurs / Threads", "Temps d'exécution", "Speedup Sp", "Efficacité Ep"
                ],
129         rows_4,
130     )
131
132     # --- Tracé des Graphiques de Benchmark ---
133     print("\n[Graphiques] Enregistrement des courbes de benchmark...")
134
135     # Figure 1 : Speedup Naïf vs MKL
136     fig, ax1 = plt.subplots(figsize=(8, 5))
137     pts = [r["n_patients"] for r in res_naive_mkl]
138     sp = [r["speedup"] for r in res_naive_mkl]
139     t_naive = [r["time_naive_ms"] for r in res_naive_mkl]
140     t_mkl = [r["time_mkl_ms"] for r in res_naive_mkl]
141
142     ax1.plot(pts, sp, marker="o", lw=2.5, color="#2b6cb0", label="Facteur d'Acc
            élération (Speedup)")
143     ax1.set_xlabel("Taille de la cohorte (Nombre de patients)", fontsize=11)
144     ax1.set_ylabel("Facteur d'accélération (x plus rapide)", color="#2b6cb0",
            fontsize=11)
145     ax1.tick_params(axis="y", labelcolor="#2b6cb0")
146     ax1.grid(True, alpha=0.3)
147     ax1.set_title("Accélération de la Simulation : Python Pur vs BLAS 3 / Intel
             MKL", fontsize=12, fontweight="bold")
148
149     fig.tight_layout()
150     fig_path1 = "figures/benchmark_naive_vs_mkl.png"
151     plt.savefig(fig_path1, dpi=300)
152     plt.close()
153     print(f"  -> Enregistré : {fig_path1}")
154
155     # Figure 2 : Scalabilité OpenMP / MKL
156     fig, (ax_sp, ax_eff) = plt.subplots(1, 2, figsize=(13, 5))
157     th = [r["threads"] for r in res_threads]
158     speedups = [r["speedup"] for r in res_threads]
159     efficiencies = [r["efficiency_percent"] for r in res_threads]
160
161     ax_sp.plot(th, speedups, marker="s", lw=2.5, color="#2c7a7b", label="Mesure
             MKL")
162     ax_sp.plot(th, th, linestyle="--", color="gray", label="Idéal linéaire (
            Speedup = p)")
163     ax_sp.set_title("Speedup Parallèle OpenMP / MKL", fontsize=11, fontweight="
            bold")
164     ax_sp.set_xlabel("Nombre de Threads", fontsize=10)
```

```
    ax_sp.set_ylabel("Speedup $S(p) = T_1 / T_p$", fontsize=10)
    ax_sp.grid(True, alpha=0.3)
    ax_sp.legend()

    ax_eff.plot(th, efficiencies, marker="^", lw=2.5, color="#d69e2e", label="
        Efficacité $E(p)$")
    ax_eff.axhline(100.0, linestyle="--", color="gray", label="Efficacité 100%"
        )
    ax_eff.set_title("Efficacité Parallèle (%)", fontsize=11, fontweight="bold"
        )
    ax_eff.set_xlabel("Nombre de Threads", fontsize=10)
    ax_eff.set_ylabel("Efficacité $E(p) = (S_p / p) \\times 100$", fontsize=10)
    ax_eff.set_ylim(0, 110)
    ax_eff.grid(True, alpha=0.3)
    ax_eff.legend()

    plt.tight_layout()
    fig_path2 = "figures/benchmark_mkl_threads.png"
    plt.savefig(fig_path2, dpi=300)
    plt.close()
    print(f"  -> Enregistré : {fig_path2}")

    print("\nCampagne de benchmark achevée avec succès !")


if __name__ == "__main__":
    main()
```

Listing B.2 – Script d'exécution des bancs d'essai et génération des graphiques de performance.

B.3 Suite de Tests Unitaires et d'Invariance Biologique (`tests/test_equivalence.py`)

```python
"""
Tests unitaires et de validation d'équivalence numérique :
1. Équivalence stricte entre l'implémentation naïve (Python pur) et matricielle (BLAS 3 / MKL).
2. Préservation des invariants biologiques (positivité des populations cellulaires et virales).
3. Cohérence et stabilité numérique des solveurs LAPACK (Cholesky vs Inversion).
"""

import pytest
import numpy as np

from src.cohort_model import (
    DEFAULT_PARAMS,
    build_cohort_parameters,
    initialize_cohort_state,
    evaluate_cohort_rhs,
    simulate_cohort_rk4,
)
from src.naive_model import (
    evaluate_single_patient_rhs,
    simulate_cohort_naive_rk4,
)
from src.lapack_analysis import (
    build_clinical_design_matrix,
    solve_ridge_cholesky_lapack,
    solve_ridge_naive_inverse,
    compute_cohort_pca_lapack,
)


def test_rhs_numerical_equivalence():
    """
    Vérifie que le second membre matriciel BLAS 3 dY/dt
    calcule exactement la même valeur que l'évaluation scalaire naïve.
    """
    n_patients = 10
    p = DEFAULT_PARAMS
    S, eta, M, A = build_cohort_parameters(n_patients=n_patients, params=p, seed=42)
    Y = initialize_cohort_state(n_patients=n_patients, S=S, params=p, seed=42)

    # 1. Évaluation matricielle vectorisée (BLAS 3)
    dY_blas = evaluate_cohort_rhs(
        Y, S, eta, M, A,
        beta_s=p["beta_s"], beta_r=p["beta_r"]
    )

    # 2. Évaluation scalaire naïve boucle par boucle
    dY_naive = np.zeros_like(dY_blas)
    for k in range(n_patients):
        res = evaluate_single_patient_rhs(
            y=Y[k].tolist(),
            s_k=float(S[k, 0]),
            eta_k=float(eta[k]),
            d=p["d"],
            delta=p["delta"],
            p=p["p"],
            c=p["c"],
```

```
            beta_s=p["beta_s"],
            beta_r=p["beta_r"],
            mu=p["mu"],
        )
        dY_naive[k] = res

    # Comparaison numérique stricte
    np.testing.assert_allclose(
        dY_blas, dY_naive,
        rtol=1e-12, atol=1e-12,
        err_msg="Discordance numérique entre le second membre BLAS 3 et la version naïve !"
    )


def test_trajectory_numerical_equivalence():
    """
    Vérifie que la trajectoire temporelle complète RK4 matricielle
    est identique au bit près à la simulation séquentielle naïve.
    """
    n_patients = 8
    days = 15.0
    dt = 0.05
    p = DEFAULT_PARAMS

    S, eta, M, A = build_cohort_parameters(n_patients=n_patients, params=p, seed=123)
    Y0 = initialize_cohort_state(n_patients=n_patients, S=S, params=p, seed=123)

    # 1. Simulation matricielle BLAS 3
    times, traj_mkl = simulate_cohort_rk4(
        Y0, S, eta, M, A,
        beta_s=p["beta_s"], beta_r=p["beta_r"],
        days=days, dt=dt, save_interval=int(days / dt)
    )
    final_mkl = traj_mkl[-1]

    # 2. Simulation naïve
    final_naive_list = simulate_cohort_naive_rk4(
        y0_list=Y0.tolist(),
        s_list=S[:, 0].tolist(),
        eta_list=eta.tolist(),
        d=p["d"],
        delta=p["delta"],
        p=p["p"],
        c=p["c"],
        beta_s=p["beta_s"],
        beta_r=p["beta_r"],
        mu=p["mu"],
        days=days,
        dt=dt,
    )
    final_naive = np.array(final_naive_list, dtype=np.float64)

    # Erreur relative maximale
    max_rel_err = np.max(np.abs(final_mkl - final_naive) / (np.abs(final_mkl) + 1e-10))
    assert max_rel_err < 1e-10, f"Erreur relative trop élevée : {max_rel_err}"


def test_biological_invariants():
    """
```

```
    Vérifie la non-négativité des compartiments cellulaires et viraux.
    """
    n_patients = 100
    p = DEFAULT_PARAMS
    S, eta, M, A = build_cohort_parameters(n_patients=n_patients, params=p, seed=777)
    Y0 = initialize_cohort_state(n_patients=n_patients, S=S, params=p, seed=777)

    times, trajectory = simulate_cohort_rk4(
        Y0, S, eta, M, A,
        beta_s=p["beta_s"], beta_r=p["beta_r"],
        days=60.0, dt=0.05, save_interval=20
    )

    # Tous les états doivent rester positifs ou nuls
    assert np.all(trajectory >= 0.0), "Violation biologique : populations négatives détectées !"


def test_lapack_cholesky_equivalence():
    """
    Vérifie que la résolution de régression Ridge via LAPACK Cholesky (DPOTRF+DPOTRS)
    fournit les mêmes coefficients que l'inversion explicite standard.
    """
    rng = np.random.default_rng(2026)
    X = rng.standard_normal((500, 8))
    y = rng.standard_normal(500)

    w_lapack, stats_lapack = solve_ridge_cholesky_lapack(X, y, alpha=1e-2)
    w_naive, stats_naive = solve_ridge_naive_inverse(X, y, alpha=1e-2)

    np.testing.assert_allclose(
        w_lapack, w_naive,
        rtol=1e-10, atol=1e-10,
        err_msg="Écart de coefficients entre Cholesky LAPACK et l'inversion explicite !"
    )
    assert abs(stats_lapack["rmse"] - stats_naive["rmse"]) < 1e-10


def test_lapack_pca():
    """
    Vérifie le calcul spectral ACP via LAPACK DSYEVD (valeurs propres décroissantes et orthogonales).
    """
    rng = np.random.default_rng(2026)
    data = rng.standard_normal((200, 5))

    projected, eigenvalues, components = compute_cohort_pca_lapack(data, n_components=2)

    # 1. Dimensions attendues
    assert projected.shape == (200, 2)
    assert eigenvalues.shape == (5,)
    assert components.shape == (5, 2)

    # 2. Valeurs propres décroissantes
    assert np.all(np.diff(eigenvalues) <= 0), "Les valeurs propres ne sont pas triées de façon décroissante !"

    # 3. Orthogonalité des composantes principales
```

```
171     identity_approx = components.T @ components
172     np.testing.assert_allclose(identity_approx, np.eye(2), atol=1e-12)
```

Listing B.3 – Validation pytest d'équivalence stricte ($\epsilon < 10^{-10}$) et invariants biologiques.